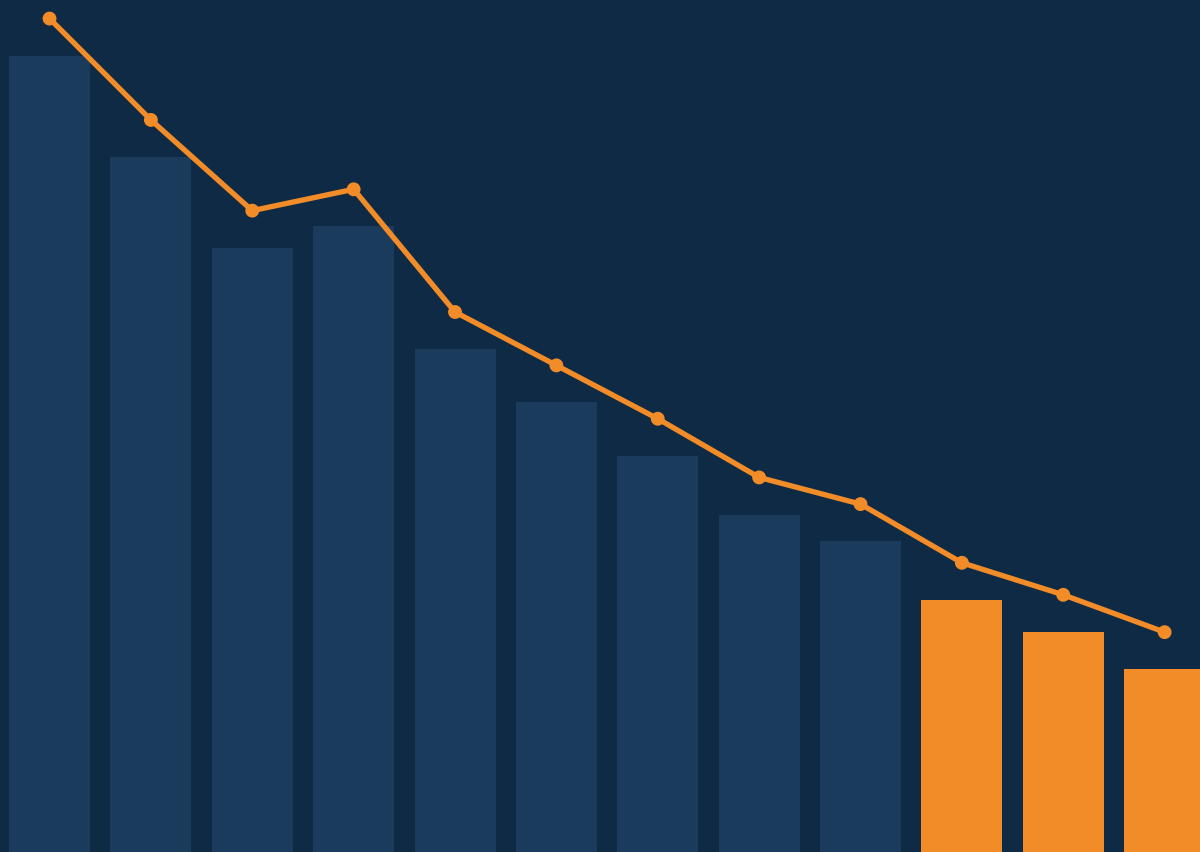


Cutting the AWS Bill

28 case studies in cost reduction:
techniques, thinking patterns and a playbook

Levels.fyi · Canva · Amazon Prime Video · Segment · Duolingo · Lyft · Airbnb
Snap · Rewind · Pinterest · Honeycomb · Zomato · Zendesk · Salesforce · Tinybird
Moneysmart · HERE · Recall.ai · DoorDash · nOps · 37signals · Dropbox
X · Prerender.io · OneUptime · GEICO



Contents

1. Executive summary	4
2. A mental model for cloud cost	8
3.A Case studies: Visibility, hygiene and reading the bill	10
CS-01 Levels.fyi, Part 1 (2024)	10
CS-02 Levels.fyi, Part 2 (September 2026)	11
CS-03 Segment (2016-2017)	12
CS-04 Duolingo (2024)	13
CS-05 Lyft (2018-2019)	15
CS-06 Airbnb	15
3.B Case studies: Storage and data lifecycle	17
CS-07 Canva	17
CS-08 Snap Inc.	18
CS-09 Rewind	18
3.C Case studies: Compute price-performance: Graviton	20
CS-10 Pinterest	20
CS-11 Honeycomb	20
CS-12 Zomato	21
CS-13 Zendesk	22
3.D Case studies: Elasticity: Spot, autoscaling and Kubernetes	23
CS-14 Salesforce Audience Studio	23
CS-15 Tinybird	23
CS-16 Salesforce (EKS fleet to Karpenter)	24
CS-17 Moneysmart Group	25
3.E Case studies: Commitments and pricing strategy	27
CS-18 HERE Technologies	27
3.F Case studies: Architecture, code and network topology	29
CS-19 Amazon Prime Video	29
CS-20 Recall.ai	30
CS-21 DoorDash	30
CS-22 nOps (its own AWS account)	31
3.G Case studies: Leaving (some of) AWS: repatriation	33
CS-23 37signals (Basecamp, HEY)	33

CS-24 Dropbox	33
CS-25 X (formerly Twitter)	34
CS-26 Prerender.io	35
CS-27 OneUptime	35
CS-28 GEICO (cautionary tale)	36
4. Cross-case patterns: how the engineers thought	38
5. Technique catalogue by AWS service	41
6. Maths and decision tools	44
7. Pitfalls and anti-patterns	47
8. A 90-day playbook and operating model	48
9. Applying this to a multi-cloud platform team	50
10. Sources and method	52

How to use this report

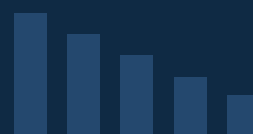
Read Sections 1 and 2 for the big picture and a vocabulary for thinking about cost. Section 3 holds the 28 case studies, grouped into seven chapters by their dominant lever; every case follows the same layout (context, what they did, results, the thinking pattern, lessons to steal, watch-outs) so you can compare them quickly. Section 4 distils the recurring thinking patterns, Section 5 is a service-by-service technique catalogue, Section 6 holds the maths, Section 7 lists pitfalls, Section 8 turns everything into a 90-day plan, and Section 9 applies it to a multi-cloud platform team's stack. Section 10 lists every source.

Reading the numbers

Savings figures come from the companies (or, where noted, from AWS or other vendors). They were not audited, and percentages rarely share a scope: '90%' for Prime Video describes one service, while '20%' for Duolingo describes the whole cloud bill. Prices quoted are approximate us-east-1 list prices as of September 2026. Treat every number as a signal of what is possible, then measure your own.

01 Executive summary

What 28 companies learned about cutting AWS spend, from a \$2,410/month clean-up to a \$74.6M storage exit.



This report studies 28 public, numbers-backed accounts of AWS cost reduction published between 2012 and September 2026. They range from a small team trimming \$2,410 a month (Levels.fyi) to Amazon's own Prime Video team cutting one service's cost by 90%, from a single log line to an exabyte-scale storage migration. The goal is not only to list techniques, but to capture how the engineers *reasoned*: what made them look, how they tested hypotheses, what they refused to trust, and what they got wrong.

28

case studies across startups, scale-ups, Amazon itself and enterprises

12

distinct cost levers identified and mapped to every case

15–90%

typical scoped reductions; most full-bill programmes land at 15-40%

\$130M+

in publicly reported savings across the dollar-quantified cases (excl. projections)

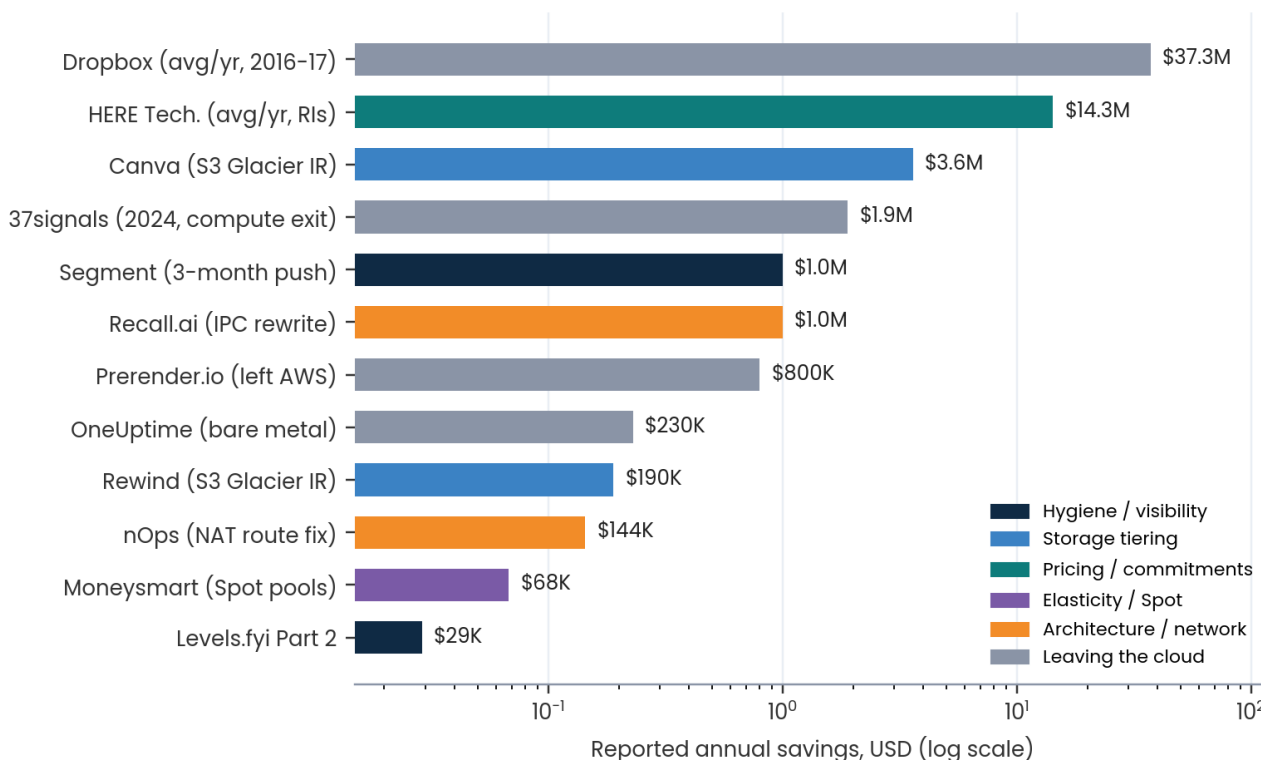
The ten findings that matter most

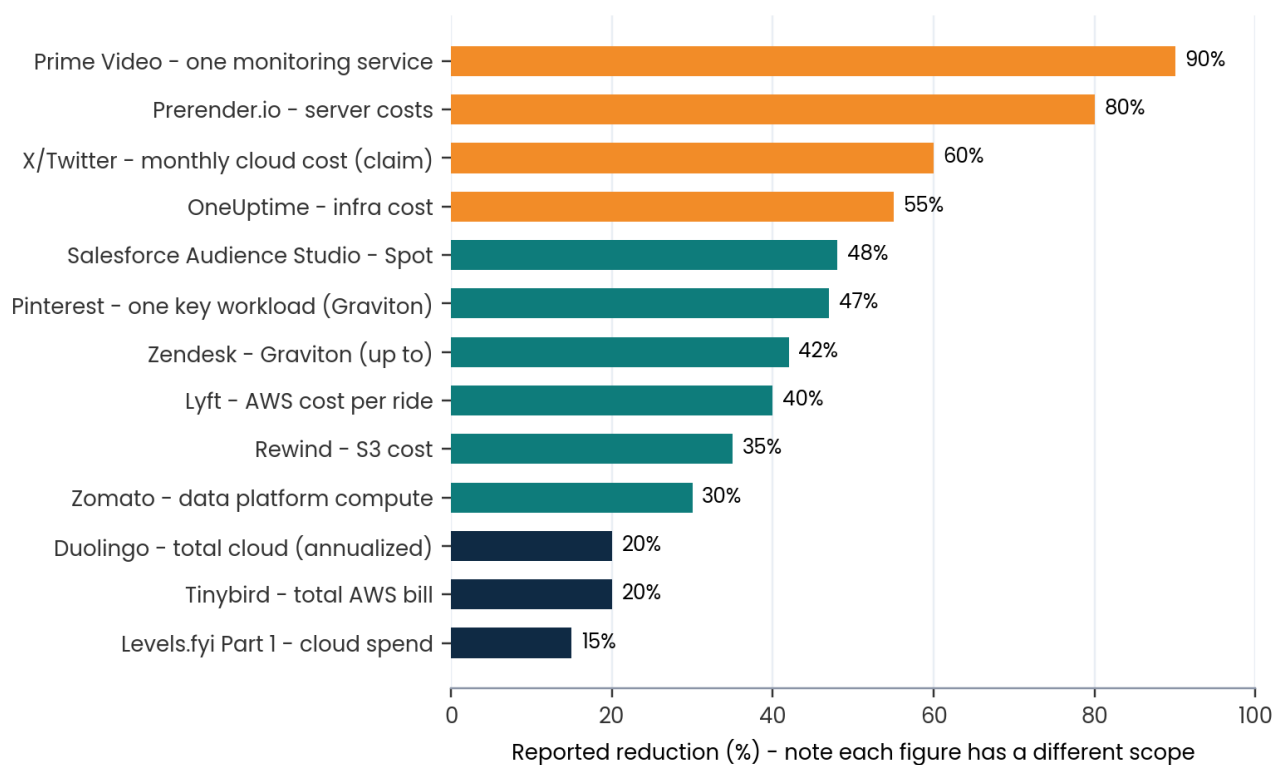
- 01 The bill is the map.** Every successful programme started from billing data, not from the architecture diagram. Segment loaded raw line items into Redshift; Levels.fyi ranked Cost Explorer line items and asked 'expected or buggy?'; Duolingo made spend queryable for every engineer.
- 02 Most waste is not a bug.** Systems were doing exactly what someone configured years earlier: backups kept for compliance, logs shipped to premium destinations, autoscaling written for old traffic, capacity raised during an incident and never lowered. Cost reviews are really reviews of old decisions.
- 03 Copies x duration is the silent multiplier.** Snapshots, object versions, logs and stale rows grow without anyone deciding they should. Levels.fyi's backups (15.7 TB for much smaller databases) and Duolingo's S3 version history are the same pattern. Retention policy is cost policy.
- 04 Per-request pricing punishes chatty designs.** Step Functions transitions per stream-second (Prime Video), internal calls hairpinning through a CDN and WAF (Levels.fyi), 2.1 billion unnecessary calls a day (Duolingo) and cross-AZ fan-out (DoorDash) all turned architecture choices into line items.
- 05 Unit economics beats absolute spend.** Lyft's cost per ride fell 40% in six months once it was measured and ranked by team. Canva's storage decision hinged on a unit ratio: savings scale with bytes, migration cost with objects.

- 06 Do the one-off versus recurring maths.** Canva paid \$1.6M to save \$3.6M a year; Rewind paid \$103K for \$16K a month; 37signals paid ~\$700K in hardware to save ~\$2M a year. Every structural change is an investment with a payback period.
- 07 Price-performance upgrades are the cheapest engineering.** Graviton delivered 30-47% savings on real workloads (Zomato, Pinterest, Zendesk) with weeks, not quarters, of effort; gp3, t4g and Aurora I/O-Optimized are similar low-risk wins.
- 08 Elasticity is a property of the workload, not the autoscaler.** Spot (Salesforce, 48%), Karpenter consolidation (Tinybird, 20% of the whole bill) and night-time shutdowns (Pinterest, Moneysmart) only work when workloads tolerate being moved or stopped.
- 09 Optimize first, then commit.** HERE saved \$50M with ~80% RI coverage, but commitments lock in today's footprint. The consistent order is: remove waste, right-size, modernize, then buy Savings Plans (now including Database Savings Plans) on the new baseline.
- 10 Know when the cloud premium buys you nothing.** Steady, storage-heavy workloads run by teams with operations skills (37signals, Dropbox, Prerender, OneUptime) saved 50-80% by leaving; lift-and-shift without modernization (GEICO) saw bills grow 2.5x. Repatriation is a valid lever at the far end of the spectrum, not a default.

Reported savings at a glance

The first chart shows annual dollar savings where companies published them (averaged per year where the source gave a multi-year total). The second shows percentage reductions. Each percentage has a different scope, from a single service (Prime Video) to a whole company's cloud bill (Duolingo), so compare them with care.





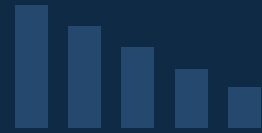
Figures are company-reported (or vendor-published where noted) and have not been independently audited. X/Twitter's dollar savings are excluded from the first chart because only third-party estimates exist.

Scoreboard: all 28 case studies

#	Company	Primary lever	Headline result
CS-01	Levels.fyi, Part 1 (2024)	Eliminate idle	~15% of total cloud spend removed
CS-02	Levels.fyi, Part 2 (September 2026)	Retention & logging	\$28.9K per year (~\$2,410/month) across four fixes
CS-03	Segment (2016-2017)	Visibility & attribution	>\$1M annual AWS savings from a three-month focused effort
CS-04	Duolingo (2024)	Visibility & attribution	20% annualized cloud savings in a few months
CS-05	Lyft (2018-2019)	Visibility & attribution	-40% AWS cost per ride within six months
CS-06	Airbnb	Visibility & attribution	CUR ingested into Airbnb's own warehouse for team-level attribution
CS-07	Canva	Storage tiering	\$3.6M saved per year (~\$300K/month), growing with data
CS-08	Snap Inc.	Storage tiering	Tens of \$M saved, with the same user experience
CS-09	Rewind	Storage tiering	-35% total S3 cost, ~6-month payback
CS-10	Pinterest	Graviton / newer gen	47% cost savings on one important workload after moving to Graviton
CS-11	Honeycomb	Graviton / newer gen	100% of the EC2 fleet migrated to Graviton
CS-12	Zomato	Graviton / newer gen	-30% compute cost, with ~25% better performance
CS-13	Zendesk	Graviton / newer gen	up to -42% cost, with up to 30% better performance
CS-14	Salesforce Audience Studio	Spot	48% overall savings after moving to Spot
CS-15	Tinybird	Spot	-20% total AWS bill (up to -90% for CI/CD workloads)
CS-16	Salesforce (EKS fleet to Karpenter)	Right-size / autoscale	~5% savings in FY2026, with a further 5-10% expected in FY2027
CS-17	Moneysmart Group	Spot	~58% net saving on Spot-managed capacity after vendor fees
CS-18	HERE Technologies	Commitments	\$50M saved plus \$150M avoided over ~3.5 years
CS-19	Amazon Prime Video	Architecture / code	-90% infrastructure cost for this service
CS-20	Recall.ai	Architecture / code	>\$1M per year saved by replacing an IPC mechanism
CS-21	DoorDash	Network path / data transfer	Cross-AZ traffic localized inside each Availability Zone
CS-22	nOps (its own AWS account)	Network path / data transfer	\$12K per month removed by fixing a single routing issue
CS-23	37signals (Basecamp, HEY)	Leave the cloud (partially)	\$10M+ projected savings over five years
CS-24	Dropbox	Leave the cloud (partially)	\$74.6M operating-expense reduction over two years
CS-25	X (formerly Twitter)	Leave the cloud (partially)	-60% monthly cloud cost (company claim)
CS-26	Prerender.io	Leave the cloud (partially)	\$1M to \$200K annual server cost (-80%)
CS-27	OneUptime	Leave the cloud (partially)	\$230K saved per year (55%+)
CS-28	GEICO (cautionary tale)	Leave the cloud (partially)	2.5x growth in cloud bills after ten years of migration

02 A mental model for cloud cost

One equation, twelve levers and an order of operations distilled from the case studies.



The cost equation

Every AWS line item is a product of a few terms. Seeing the bill this way turns vague 'cost cutting' into specific questions about which term to attack.

$$\text{Spend} = \text{Quantity} \times \text{Unit price} \times \text{Time} \times \text{Topology}$$

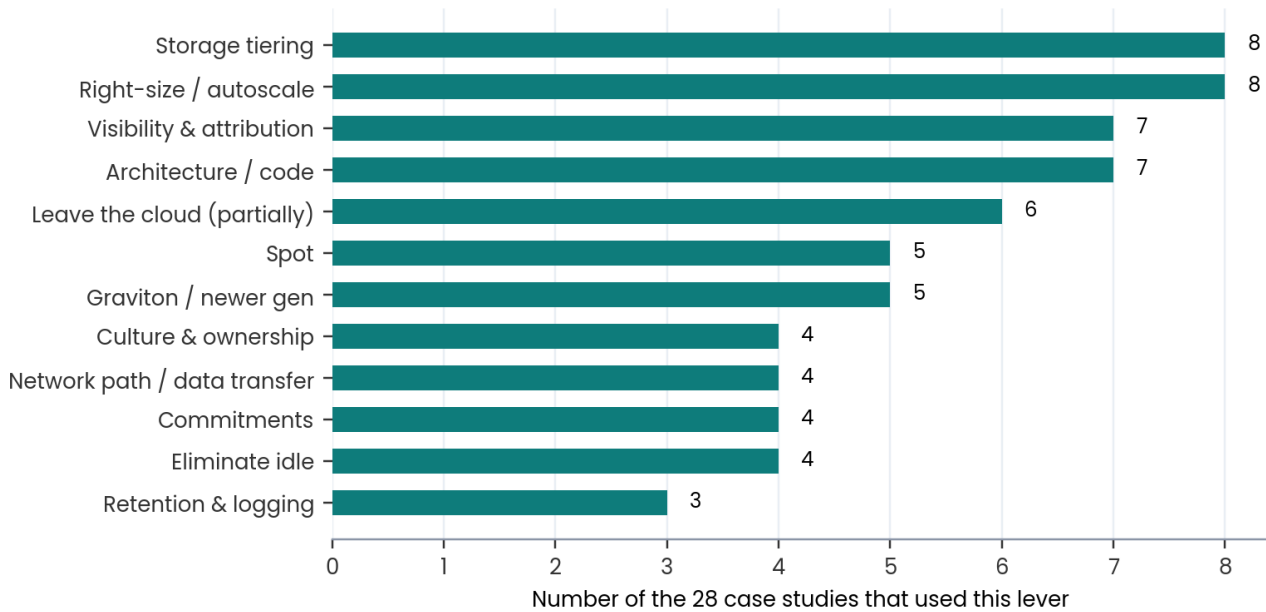
Term	What it means	How to attack it	Seen in
Quantity	How many instances, GB, requests, log lines, snapshots or state transitions you consume.	Delete idle resources; right-size; shorten retention; send fewer requests; log less; cache longer.	Levels.fyi, Segment, Duolingo
Unit price	What AWS charges per unit, which depends on the product, class, generation and purchase model.	Graviton and newer generations; cheaper storage classes; Spot; Savings Plans and RIs; pricing options such as Aurora I/O-Optimized or DynamoDB Standard-IA.	Canva, Pinterest, HERE, Salesforce
Time	How long the resource exists or the data is kept.	Autoscaling to the traffic curve; schedules for non-production; TTLs and lifecycle rules.	Pinterest 2012, Moneysmart
Topology	The path data takes: across AZs, through NAT, out to the internet and back, through per-request edge services.	Gateway and interface endpoints; zone-aware routing; PrivateLink; avoid hairpins.	DoorDash, nOps, Levels.fyi
Venue	Where the workload runs at all.	Keep in AWS, move to a cheaper provider, or run on owned hardware.	37signals, Dropbox, OneUptime

Choose a unit metric before you start

Absolute spend should rise with a growing business. What must fall is cost per unit of value: per ride (Lyft), per event processed (Segment), per active learner, per GB ingested, per thousand page views. Pick one, chart it weekly, and make every optimization report its effect on that number.

The twelve levers

Across the 28 cases, every action fits one of twelve levers. The chart counts how many cases used each one. Visibility and architecture/code changes are the most common, which says something important: the biggest wins rarely come from a pricing trick alone.



Order of operations

The cases converge on a sequence. Doing the steps out of order is the most common way to waste effort, for example buying three-year commitments for instances that right-sizing would have halved.

Step	Verb	What it involves
1	See	Make spend queryable and attributable (CUR 2.0 / Data Exports, tags, per-team views, anomaly detection).
2	Stop	Delete what nobody needs: idle instances, orphaned volumes and caches, forgotten environments.
3	Shrink	Right-size instances, containers and databases; shorten retention; cut log volume.
4	Swap	Move to better price-performance: Graviton, gp3, newer generations, cheaper storage classes.
5	Flex	Match capacity to demand: autoscaling, schedules, Spot, Karpenter consolidation.
6	Redesign	Fix chatty paths, hairpins, cross-AZ fan-out and per-request inner loops.
7	Commit	Buy Savings Plans / RIs / Database Savings Plans on the new, smaller baseline.
8	Relocate	For steady, storage-heavy workloads, compare against other venues on full cost of ownership.

Four shapes of spend

Before choosing tools, classify each large line item by its shape over time. The shape tells you which lever fits.

Shape	Typical examples	Best-fit levers	Seen in
Flat baseline	Databases, core services, always-on clusters.	Right-size, Graviton, then commit (SP/RI/DB SP).	HERE, Zomato
Periodic	Web tiers with day/night cycles, office-hours non-production.	Autoscaling, schedules.	Pinterest 2012, Moneysmart
Spiky / batch	ETL, CI/CD, analytics jobs, ML training.	Spot, Karpenter, serverless for low duty cycles.	Salesforce, Tinybird, Lyft CI
Growing with data	S3, snapshots, logs, DynamoDB tables.	Lifecycle, tiering, TTLs, retention policy.	Canva, Snap, Duolingo

A Visibility, hygiene and reading the bill

Six cases where the main tool was attention: making spend visible, questioning each line item and cleaning up.



These cases share a method more than a technology. Each team turned billing data into something it could query, ranked what it found, and questioned whether each large number was expected. The savings range from a few thousand dollars a month to over a million a year, but the thinking is the same, and it is the foundation every other chapter builds on.

CS-01

Levels.fyi, Part 1 (2024)

An 'inverted treasure hunt' through Cost Explorer

Business	Compensation-data platform
Scale	~650M page views, ~26M visitors in 12 months
Stack	CloudFront, ECS Fargate, RDS, Lambda, ElastiCache, S3
Published	June 2024

~15%

of total cloud spend removed

CONTEXT

Levels.fyi runs a lean, AWS-native stack. When the team opened Cost Explorer they found that roughly a quarter of the bill was CloudFront, with the rest spread across compute, databases, caches and storage. Nothing was on fire; the goal was simply to stop paying for things that no longer earned their keep.

WHAT THEY DID

- ▶ **Deleted forgotten resources.** EC2 instances that had been spun up temporarily so employees could reach internal systems were still running long after the need passed. They were removed together with their attached EBS volumes.
- ▶ **Verified before deleting.** An ElastiCache cluster was referenced by no service, yet CloudWatch still showed CPU and network activity. Instead of trusting the graph, the team connected to it and checked DBSIZE and INFO (connected clients, total commands processed). The 'activity' turned out to be the backup job. One final backup was taken, then the cluster was deleted.
- ▶ **Tiered cold data.** Terabytes of historical job data in S3 were moved to Glacier Deep Archive with a lifecycle rule. Because S3 event notifications fed an SQS pipeline, a mass storage-class change would have flooded the backend with events, so the listeners were detached for the migration and re-attached afterwards.
- ▶ **Re-tuned autoscaling.** Fargate scaling rules written for an older traffic shape kept several tasks alive during quiet periods; they were re-fitted to current traffic.
- ▶ **Moved to newer, cheaper hardware.** RDS went from t3 to Graviton-based t4g instances and from gp2 to gp3 volumes (after backups, in a low-traffic window). Lambda functions were moved to arm64 after checking dependency compatibility.
- ▶ **Retention hygiene.** An ECR lifecycle rule now keeps only the last 25 image versions, CloudWatch log retention was shortened, and load balancers and NAT gateways were audited for anything unused.
- ▶ **Commitments.** Reservations plus a third-party group-buying service trimmed roughly \$700 per month from EC2 and ECS.

Results

- ✓ About 15% of total cloud spend eliminated without a performance or reliability trade-off.
- ✓ Next target identified: replacing CloudFront with a cheaper CDN, since the CDN was the single biggest line item.

The thinking pattern

Start from the bill, not from the architecture diagram. Treat every line item as a claim that must be proven ('this resource is needed'), and prove it from inside the resource rather than from surface metrics. Before changing anything that is wired into an event-driven system, ask what side-effects the change itself will emit.

LESSONS TO STEAL

- Anything created 'temporarily' needs an owner tag and an expiry date, or it becomes permanent spend.
- Activity is not usage: backups, health checks and replication generate metrics on resources nobody uses.
- Storage-class transitions can fire S3 events; pause or filter consumers before bulk transitions.
- Graviton instance classes and gp3 volumes are low-risk, high-certainty wins for managed databases.

Watch-outs and caveats. Pooled or group-bought commitments depend on AWS's resale terms, which were reportedly tightened in 2025 to stop sharing RIs and Savings Plans across different end customers. Check how any such service is structured before relying on it.

Levers: Eliminate idle · Right-size / autoscale · Storage tiering · Graviton / newer gen · Commitments · Retention & logging **Sources:** levels.fyi/blog/slashing-aws-bill.html

CS-02**Levels.fyi, Part 2 (September 2026)***Four fixes, \$28.9K a year, and nothing switched off*

Business	Compensation-data platform
Edge traffic	~155M requests per month
Method	Largest line items first; LLM-assisted forensics
Published	22 Sept 2026

\$28.9K

per year (~\$2,410/month) across four fixes

CONTEXT

Two years after the first clean-up, the team returned with a different question. Instead of hunting for idle resources, they took each large line item in Cost Explorer and asked: is this number expected, or does something feel wrong? To move fast, they gave an LLM read-only AWS credentials and let it query Cost Explorer, CloudWatch and service APIs through the AWS CLI, then pointed it at questions with measurable answers, such as which log groups were growing fastest and what was writing to them, or what was pushing RDS cost up by a small amount every single day.

WHAT THEY DID

- ▶ **Backups: from 15.7 TB to 3 TB.** SOC 2 preparation had produced daily RDS snapshots on a long, flat retention. Backup storage had quietly become most of the RDS bill, far exceeding the size of the databases themselves. Re-reading the requirement showed that SOC 2 asks for a defined retention period that you actually follow, not a maximal one. A tiered policy (daily for a month, weekly for a few months, monthly for a year, a few long-term copies) cut storage from 15.7 TB to 3 TB and saved about \$1,200 per month while keeping point-in-time recovery for recent days.
- ▶ **WAF and CDN logs: \$640 to under \$70.** Logs were shipped to one of AWS's most expensive destinations (priced well above S3) and rarely read, and they included every allowed request. Moving them to cheaper storage and keeping only the events actually reviewed (chiefly blocked traffic) saved about \$600 a month.
- ▶ **One line of logging.** A small line item kept creeping upward: \$26, then \$36, then \$41. The cause was a single statement in the shared Lambda entry point that logged the entire incoming event on every invocation. A compact summary line replaced it (about 99% smaller). A second function logged one line per item inside a loop over hundreds of thousands of items; info-level logging there was simply

removed, keeping errors only.

- **Stopped paying tolls to talk to itself.** Most edge requests were not users at all but the Next.js server-side renderer calling the company's own API through the public CDN and WAF, then back into the same AWS account. Two obstacles blocked a simple fix: the frontend and backend VPCs used the same CIDR range (so peering was impossible), and the CDN was absorbing about three-quarters of those calls as cache hits. The team first added a Valkey cache next to the frontend and ran it while the CDN was still in path; it reached a 78.8% hit rate versus the CDN's 77.3%. Only then did they switch backend calls onto AWS PrivateLink, which works across overlapping address ranges.

Results

- ✓ Internal traffic through the public edge fell by more than 99%; CloudFront requests dropped ~73%, WAF requests ~72%, and the frontend NAT gateway lost roughly three-quarters of its traffic.
- ✓ Net saving of ~\$549/month after paying for the new pieces (PrivateLink costs about \$36/month).
- ✓ Side benefit: the internal hop became roughly three times faster (18 ms to 6 ms), and WAF rules now reason only about genuinely public traffic.
- ✓ Total: about \$2,410/month, or ~\$28,900/year.

The thinking pattern

Nothing here was a bug. Backups, logs and routing were all doing exactly what they had been configured to do. Cost work at this stage is about re-reading old decisions (compliance-driven retention, default log destinations, debug logging, convenient network paths) and asking whether they still match what the business wants. The team also watched slopes, not just levels: a line item that grows a little every day is a lead worth chasing even when it is small.

LESSONS TO STEAL

- Compliance usually requires retention that is defined and followed, not retention that is maximal. Grandfather-father-son rotation keeps recovery points while cutting copies.
- Anything priced per request (CDN, WAF, NAT, API Gateway) punishes internal traffic that hairpins through public edges.
- Shadow-run a replacement cache while the old path is still live, so an under-performer fails harmlessly.
- An LLM with read-only credentials is an effective cost-forensics assistant when asked precise, answerable questions.
- Cost Explorer's latest days are incomplete. Validate with a 'control' line item that should not have moved, and with real-time signals such as logs.

Watch-outs and caveats. Three near-misses are worth copying as checklist items. A 50 ms cache timeout tripped a circuit breaker 21 times in ten minutes with no real errors, because during server rendering it was measuring how long the busy process took to notice the reply (and it threw away reads that had already succeeded). A proof of concept tested one instance and missed that half the fleet sat in an Availability Zone the new path did not cover. And after the swap, backend load briefly rose because the new cache's TTL cap was shorter than the CDN's.

Levers: Retention & logging · Network path / data transfer · Architecture / code · Visibility & attribution **Sources:** levels.fyi/blog/slashing-our-aws-bill-at-levelsfyi-part-2.html

CS-03

Segment (2016–2017)

The million-dollar engineering problem

Business	Customer-data infrastructure (now Twilio Segment)
Load	Tens of thousands of events per second
Team	Two engineers, ~3 months
Published	March 2017

>\$1M

annual AWS savings from a three-month focused effort

CONTEXT

Most SaaS products get economies of scale early, because one server can serve many users. Segment's costs grew linearly with adoption instead: every customer event had to be processed. By the end of Q3 2016, the AWS bill made up two-thirds of cost of goods sold, and it was rising faster than usage.

WHAT THEY DID

- ▶ **Made the bill queryable.** They enabled AWS detailed billing, which drops raw line items into S3, and loaded them into Redshift for analysis. Line-by-line review surfaced about fifteen problem areas that together made up roughly 40% of the monthly bill.
- ▶ **Cleared incident leftovers.** Some findings were mundane: hundreds of large EBS volumes and over-provisioned cache and RDS instances that had been scaled up during past incidents and never scaled back down.
- ▶ **Fixed DynamoDB hot shards (~\$300K/yr).** Provisioned throughput had been raised table-wide to satisfy a handful of very hot partition keys. Fixing the access pattern removed the need to pay for capacity the rest of the table never used.
- ▶ **Service auto-scaling (~\$60K/yr).** Services that ran at fixed capacity were put on auto-scaling.
- ▶ **Bin-packing and fewer instance types (~\$240K/yr).** Consolidating onto fewer, better-packed instance types raised utilization.

Results

- ✓ Over \$1M per year removed from the AWS bill.
- ✓ A repeatable method: raw billing data, SQL, a ranked list of problems, then engineering investment where it pays.

The thinking pattern

The authors were blunt that no tool is a magic bullet; the work is digging through the bill line by line. They also framed cost as a business constraint (gross margin), not a housekeeping chore, which justified dedicating engineers to it.

LESSONS TO STEAL

- Put billing data in a warehouse. Today that means CUR 2.0 / Data Exports queried with Athena or your own warehouse.
- Every incident scale-up should create a follow-up ticket to scale back down.
- For DynamoDB, capacity is set by the hottest key. Fix key design before buying throughput.
- A smaller menu of instance types packs better and is easier to commit to.

Watch-outs and caveats. The specific tooling (detailed billing reports, the Redshift loader) has been superseded by CUR 2.0 and Data Exports, but the approach is unchanged.

Levers: Visibility & attribution · Eliminate idle · Right-size / autoscale · Architecture / code **Sources:** segment.com/blog/the-million-dollar-eng-problem; informationweek.com (interview with Segment CTO)

CS-04

Duolingo (2024)

Cost optimization and code health go hand in hand

Business	Language-learning app, 100+ microservices
Trigger	New AI-heavy features drove spend into millions/yr
Effort	Dozens of engineers, a few months
Published	October 2024

20%

annualized cloud savings in a few months

CONTEXT

Features like Stories, DuoRadio and Adventures each added infrastructure. At the start of 2024 Duolingo launched a company-wide effort to cut cloud spend without hurting the product. A five-person FinOps team supported the engineers.

WHAT THEY DID

- ▶ **Visibility first.** A cost platform (CloudZero) broke spend into queryable items. One early surprise: a service's staging environment cost more than production because someone had scaled it up for a test and never scaled it back. Cost data was then wired into the normal metrics stack and weekly per-team reports, and later extended to non-AWS spend such as LLM APIs.
- ▶ **Deleted what was not needed.** Old ElastiCache clusters, whole databases and even an entire microservice belonging to retired features were removed.
- ▶ **Deleted data, not just resources.** S3 buckets in active use were paying to keep their complete version history, so lifecycle rules were added. DynamoDB tables held stale rows because TTLs were missing. Production logging was verbose and unsampled; stack traces in particular were expensive.
- ▶ **Right-sized memory and task counts.** Most services were over-provisioned. Testing showed nearly all could run at 90-95% memory utilization. Minimum task counts were lowered and scale-out made less aggressive; one service alone saved hundreds of thousands of dollars a year.
- ▶ **Used built-in pricing options.** Switching a single database to Aurora I/O-Optimized saved several hundred thousand dollars a year; DynamoDB Standard-IA table class was used where it fit; RI purchases were sized to the measured baseline.
- ▶ **Reduced traffic.** Refactoring a legacy service that triggered side effects eliminated about 2.1 billion unnecessary internal calls per day. Raising a cache TTL from one minute to one hour for a rarely changing resource cut traffic to another service by over 60%.

Results

- ✓ 20% annualized reduction in cloud spend within a few months (absolute amount undisclosed).
- ✓ Cleaner codebase: many savings came from removing tech debt and simplifying code.

The thinking pattern

Make cost a first-class engineering signal. When owners can see the literal price of their tech debt, they make different choices. Duolingo pushed cost into design documents (cost estimates in tech specs) and into regular team rituals (quarterly spend reviews), and automated anomaly detection across services rather than relying on manual monitoring.

LESSONS TO STEAL

- Look for staging or test environments that cost more than production.
- Adding a DynamoDB TTL does not clean up existing rows; backfill the TTL attribute or run a one-off purge.
- Stack traces and unsampled debug logs are some of the most expensive bytes you will ever write.
- Microservice fan-out multiplies load-balancer, cross-AZ and task costs; fix chatty call paths and cache stable data longer.

- Aurora I/O-Optimized can be dramatically cheaper for I/O-heavy clusters (AWS suggests evaluating it when I/O exceeds about a quarter of Aurora spend).

Watch-outs and caveats. Tools matter less than coverage: the win came from getting many engineers looking at cost data they could act on.

Levers: Visibility & attribution · Eliminate idle · Right-size / autoscale · Retention & logging · Commitments · Architecture / code · Culture & ownership Sources: blog.duolingo.com/reducing-cloud-spending; blog.duolingo.com/finops; infoq.com (Oct 2025)

CS-05

Lyft (2018–2019)

Unit economics: AWS cost per ride

Business	Ride-hailing
Commitment	>= \$300M on AWS for 2019-2021 (IPO filing)
Growth	~50M rides (2015) to 350M+
Presented	AWS re:Invent 2019

-40%

AWS cost per ride within six months

CONTEXT

Hyper-growth had outpaced cost management. The absolute bill was expected to grow with rides, so the useful question was whether each ride was getting cheaper to serve.

WHAT THEY DID

- ▶ **Version 1: a spreadsheet and a script.** A Python script divided AWS spend by the number of rides and ranked engineering teams by their spend trend. The ranking had a gamification effect: teams competed to improve.
- ▶ **Version 2: a proper data pipeline.** Because discounts and Reserved Instance costs were not always reflected transparently in the raw usage reports, Lyft built a pipeline into a data warehouse with a reporting and dashboard layer that could be queried in SQL.
- ▶ **Targeted engineering.** The finer-grained data led to halving read/write spend on the top 25 DynamoDB tables and to ~50% savings tied to Kubernetes/container migration work.
- ▶ **Spot for CI.** In a separate AWS case study, Lyft cut continuous-delivery infrastructure costs by up to 75% by moving it to Spot Instances, reportedly with a very small configuration change.

Results

- ✓ Cost per ride down 40% in six months, while ride volume grew.
- ✓ Top DynamoDB tables' read/write spend cut in half.

The thinking pattern

Normalize cost by the business driver. A rising bill can be healthy; a rising cost per unit never is. Then make that number visible per team so ownership is obvious and peer pressure does some of the work.

LESSONS TO STEAL

- Pick one unit metric (cost per ride, order, active user, GB ingested, API call) and chart it weekly.
- Allocate discounts (RIs, Savings Plans, credits) correctly or teams will optimize against misleading numbers.
- CI/CD fleets are among the safest places to start with Spot.

Watch-outs and caveats. Team leaderboards work best when the metric is fair; normalize for teams whose costs are driven by other teams' traffic.

Levers: Visibility & attribution · Culture & ownership · Spot · Architecture / code Sources: techtarget.com (re:Invent 2019 coverage); casestudies.com / AWS Lyft Spot case study

CS-06

Airbnb*Treat billing as a data product*

Business	Travel marketplace, early AWS adopter
Focus	Attribution, commitments, storage tiering
Source type	AWS case study (as summarized by third parties)
Era	2020-2022

CUR

ingested into Airbnb's own warehouse
for team-level attribution

CONTEXT

At Airbnb's scale, the challenge is less about finding single idle servers and more about making thousands of engineers aware of what their services cost.

WHAT THEY DID

- ▶ **Own the cost data.** Airbnb uses Cost Explorer for quick views but loads the Cost and Usage Report into its internal data infrastructure, so spend can be sliced hourly, daily or monthly and attributed to the teams that own it.
- ▶ **Savings Plans instead of instance-by-instance RIs.** Moving EC2 commitments to Savings Plans reduced the operational effort of managing commitments while still producing meaningful savings.
- ▶ **Intelligent-Tiering for uncertain access.** S3 Intelligent-Tiering is used where access patterns are hard to predict.

Results

- ✓ Lower commitment-management overhead and team-level cost accountability (dollar figures not published).

The thinking pattern

Cost attribution is a data engineering problem. Once billing data sits next to business data, the same analytics stack that powers the product can power cost decisions.

LESSONS TO STEAL

- Build on CUR / Data Exports early; a warehouse beats any console for attribution.
- Prefer Savings Plans for flexibility when your instance mix changes often.
- Intelligent-Tiering charges a small per-object monitoring fee and does not auto-tier objects under 128 KB, so it suits buckets of larger objects with unpredictable access.

Watch-outs and caveats. Details here come from AWS's published case study as summarized by third parties; treat specifics as directional.

Levers: Visibility & attribution · Commitments · Storage tiering · Culture & ownership **Sources:**
aws.amazon.com/solutions/case-studies/airbnb-optimizes-usage-and-costs-case-study; hystax.com summary

B Storage and data lifecycle

Three cases where matching storage class to real access patterns saved millions, and one where the migration itself cost six figures.



Storage bills grow quietly because data rarely gets deleted. These cases show how to match storage class to actual access patterns, and how to price the migration itself, which is often the part teams forget.

CS-07

Canva

Bytes pay the bill, objects pay the migration

Business	Online design tool, 100M+ MAU
Data	230+ PB in S3; largest bucket ~45 PB
Change	Standard-IA to S3 Glacier Instant Retrieval
Published	AWS Storage Blog

\$3.6M

saved per year (~\$300K/month), growing with data

CONTEXT

Canva keeps templates, stock photos and graphics in S3 Standard because users hit them constantly. User uploads and designs behave differently: they are accessed heavily for a short time while a design is being made, then rarely. Those had historically lived in Standard-IA. Logs and backups used Glacier Flexible Retrieval.

WHAT THEY DID

- **Measured access patterns.** Using S3 Storage Class Analysis and internal frequency-segmentation work, Canva saw retrievals drop sharply after about 15 days, while IA data kept a flat, low retrieval rate over time.
- **Waited for the right product.** S3 Glacier Instant Retrieval (launched late 2021) offered archive-level storage prices with millisecond access, which fit user content that is rarely opened but must open instantly.
- **Did the migration math per bucket.** Transitions are billed per object, while savings accrue per byte. Moving everything would have cost millions (a third-party write-up puts it near \$6M). So Canva computed break-even time by each bucket's average object size and prioritized buckets whose average object was roughly 400 KB or larger.
- **Moved selectively.** About 80 billion objects were transitioned for a one-off cost of about \$1.6M.

Results

- ✓ Roughly 130 PB of 230 PB now sits in Glacier Instant Retrieval.
- ✓ About \$300K saved per month (\$3.6M per year), with positive ROI after only a few months.
- ✓ Savings keep growing because user-generated data keeps growing.

The thinking pattern

This is unit economics applied to storage. Ongoing savings scale with bytes; one-time transition cost scales with object count. Their ratio is the average object size, which makes it the single best predictor of payback. Rank by that, and migrate the fastest-payback buckets first.

LESSONS TO STEAL

- Measure access before choosing a class: S3 Storage Class Analysis and Storage Lens are cheap ways to do it.

- Compute payback per bucket (see Section 6 for the formula and a ready-made table).
- Glacier IR has a 90-day minimum storage duration and a 128 KB minimum billable object size; retrievals cost more than IA.
- Re-check your storage map whenever AWS launches a new class. New price points change the optimal answer.

Watch-outs and caveats. Glacier IR is only a win if retrievals really are rare. A product change that makes old content popular again can reverse the economics, so keep monitoring retrieval volume.

Levers: Storage tiering · Visibility & attribution **Sources:**

aws.amazon.com/blogs/storage/how-canva-saves-over-3-million-annually-in-amazon-s3-costs; urielbitton.substack.com

CS-08

Snap Inc.

Two exabytes of Memories on Glacier Instant Retrieval

Business	Snapchat; 363M+ DAU at the time
Data	2+ EB, ~1.5 trillion photos and videos
Change	Long-lived media to S3 Glacier IR
Source	AWS case study, 2022

Tens of \$M

saved, with the same user experience

CONTEXT

Snapchat began as an ephemeral app, but features like Memories turned it into a long-term media store. Most saved media is rarely re-opened, yet it must load instantly when it is.

WHAT THEY DID

- ▶ **Matched storage class to product semantics.** Snap migrated its storage from another cloud provider to S3 and placed long-term, rarely accessed media in S3 Glacier Instant Retrieval.
- ▶ **Migrated invisibly.** More than 2 EB (roughly 1.5 trillion files) moved without users noticing.

Results

- ✓ Tens of millions of dollars saved.
- ✓ 99.99% availability; 20-30% lower download latency in some regions.

The thinking pattern

Start from the product promise (instant access, rare access) and pick the cheapest storage that keeps that promise.

LESSONS TO STEAL

- Translate product features into access SLOs; the SLO, not habit, should choose the storage class.
- At exabyte scale, even a few tenths of a cent per GB-month is tens of millions of dollars a year.

Watch-outs and caveats. Snap's move also involved a cross-cloud migration; its economics reflect negotiated pricing at enormous scale.

Levers: Storage tiering **Sources:** aws.amazon.com/solutions/case-studies/snap-case-study

CS-09

Rewind

A lifecycle-rule edit with a \$103K invoice attached

Business	SaaS backup provider
Data	Tens of billions of small objects
Change	Standard-IA to Glacier IR via lifecycle rule
Source	Rewind engineering blog

-35%

total S3 cost, ~6-month payback

CONTEXT

A backup company stores huge numbers of objects that are almost never read, which makes it a textbook Glacier IR candidate.

WHAT THEY DID

- ▶ **Priced the transition first.** Moving the content would cost about \$103,000 in one-off transition requests against roughly \$16,000 of monthly savings, a payback of a little over six months. They accepted it and amortized the cost.
- ▶ **Respected the 90-day minimum.** Glacier IR bills at least 90 days of storage per object. The lifecycle transition was set at 90 days so objects deleted earlier would not incur early-deletion charges.
- ▶ **Let S3 do the work.** After the rule change, S3 transitioned the eligible objects in about two days.

Results

- ✓ Overall S3 cost dropped about 35% while stored bytes stayed flat.

The thinking pattern

A single configuration change can produce a large one-time bill. Estimate the one-off cost and the payback period before you save the lifecycle rule, then decide deliberately.

LESSONS TO STEAL

- Payback = one-off transition cost / monthly saving. Write it down before you flip the switch.
- Align transition ages with minimum storage durations to avoid early-deletion fees.
- Small objects are expensive to move; consider aggregating them before archiving.

Watch-outs and caveats. Rewind's objects were small, which made the transition expensive relative to savings; with larger objects the payback would have been much faster (compare Canva).

Levers: Storage tiering **Sources:** rewind.com/blog/saving-money-aws-glacier-instant-retrieval

C Compute price-performance: Graviton

Four cases where changing CPU architecture cut compute cost by 30–47% with modest engineering effort.



When a newer instance family is both faster and cheaper, migrating pays twice: you need fewer instances and each one costs less. Graviton is the clearest example on AWS, and these cases show how companies de-risked the move.

CS-10

Pinterest

From night-time shutdowns in 2012 to a Graviton program in 2021–2024

Business	Visual discovery, 500M+ MAU
Data	Nearly an exabyte in S3
Programs	Autoscaling (2012), Graviton (2021+), S3 lifecycle
Source	AWS case studies, HN

47%

cost savings on one important workload after moving to Graviton

CONTEXT

Pinterest was born on AWS and has run several distinct efficiency programs as it grew.

WHAT THEY DID

- **2012: follow the traffic curve.** An autoscaling system for the web application pool, built in about two weeks, cut the pool's hourly cost from about \$54 to about \$20 by shrinking capacity at night.
- **2021 onward: Graviton migration program.** Workloads were moved to Graviton-based EC2 instances. On one important workload this delivered 38% compute-resource savings, 47% cost savings and 62% lower carbon emissions. By 2024 more than a quarter of the infrastructure had moved, with similar results.
- **Storage lifecycle at exabyte scale.** S3 Lifecycle rules assign data to the right storage classes, including Glacier Deep Archive for data that must be kept but is almost never read.

Results

- ✓ Large, repeatable compute savings plus a measurable sustainability benefit.

The thinking pattern

Treat efficiency as a multi-year program rather than a one-off project, and pair cost with another goal (here, carbon) so the program keeps sponsorship.

LESSONS TO STEAL

- Time-of-day elasticity is still one of the simplest big wins for stateless tiers.
- Prove Graviton on one important workload, then generalize with the same playbook.
- Sustainability metrics are a useful second justification for efficiency work.

Watch-outs and caveats. The 47% figure is for a single workload; fleet-wide savings will usually be lower than the best case.

Levers: Graviton / newer gen · Right-size / autoscale · Storage tiering **Sources:** aws.amazon.com/solutions/case-studies/pinterest-graviton-case-study; news.ycombinator.com (Ryan Park, 2013)

CS-11

Honeycomb*Invest once in multi-arch, then move everything*

Business	Observability platform
Milestone	100% of EC2 fleet on Arm64 (2022)
Coverage	Spot, C/M/R families, NVMe variants, RDS, ElastiCache
Source	Honeycomb blog

100%

of the EC2 fleet migrated to Graviton

CONTEXT

Honeycomb ingests and queries large volumes of telemetry, so compute is its dominant cost.

WHAT THEY DID

- ▶ **A one-week bake-off.** An engineer's side project ran Honeycomb on Graviton2 M6g instances for a week and saw real gains in both performance and cost.
- ▶ **Own code first.** Services Honeycomb builds itself were the easiest to move (just recompile for arm64) and also the largest share of compute spend, so they went first.
- ▶ **Then the long tail.** Once multi-architecture builds existed, the team kept going through third-party and managed components until the whole EC2 fleet ran on Arm.

Results

- ✓ Entire EC2 fleet on Graviton; Arm has publicized a roughly 50% infrastructure-cost reduction for Honeycomb (vendor figure).

The thinking pattern

The expensive part of an architecture migration is building the capability (multi-arch images, arm CI runners, benchmarks). Once paid for, the marginal cost of moving another workload is small, so do not stop after the first one.

LESSONS TO STEAL

- Build multi-arch images (for example with docker buildx) and run CI on arm64 as well as x86.
- Benchmark with production-like load; price-performance, not list price, is the real gain.
- Watch for native dependencies and x86-only binaries in the long tail.

Watch-outs and caveats. The 50% figure comes from Arm's marketing materials rather than a detailed Honeycomb cost breakdown.

Levers: Graviton / newer gen **Sources:** honeycomb.io/blog/graviton-migration; newsroom.arm.com

CS-12

Zomato*A two-week Graviton move for Trino and Druid*

Business	Food delivery (India)
Workload	Java-based Trino and Apache Druid analytics
Duration	About two weeks including testing
Source	AWS Open Source Blog

-30%

compute cost, with ~25% better performance

CONTEXT

Zomato's data platform runs near-real-time analytics on open-source Java engines, which are usually portable across CPU architectures.

WHAT THEY DID

- **Migrated the analytics clusters.** Trino and Druid were moved to Graviton-based instances with only minor tweaks.

Results

- ✓ Compute cost down about 30%; performance up about 25%; the whole effort, testing included, took about two weeks.

The thinking pattern

Pick the workload with the best ratio of spend to migration risk. JVM-based, horizontally scaled data systems usually top that list.

LESSONS TO STEAL

- Data platforms (Trino, Druid, Spark, Kafka, OpenSearch) are excellent first Graviton candidates.
- Run old and new node pools side by side and compare query latency before cutting over.

Watch-outs and caveats. Results depend on JVM version; newer JDKs have much better arm64 optimizations.

Levers: Graviton / newer gen **Sources:** aws.amazon.com/blogs/opensource (Zomato Trino/Druid on Graviton); aws.amazon.com/ec2/graviton (customer quotes)

CS-13**Zendesk**

Graviton as a fleet-wide price-performance lever

Business	Customer-service SaaS
Change	Migration to Graviton-based instances
Source	AWS Graviton customer page
Result type	Up-to figures

up to -42%

cost, with up to 30% better performance

CONTEXT

Zendesk is a representative example of a mature SaaS company treating CPU architecture as a cost lever.

WHAT THEY DID

- **Moved workloads to Graviton.** Workloads were migrated to Graviton-based instances.

Results

- ✓ Up to 30% better performance and up to 42% lower cost, plus sustainability benefits.

The thinking pattern

When a newer generation offers better performance at a lower price, the migration pays twice: fewer instances, each cheaper.

LESSONS TO STEAL

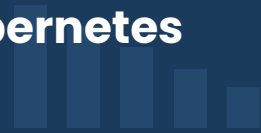
- Measure savings per workload; 'up to' numbers describe the best cases.
- Pair Graviton moves with right-sizing, since better per-core performance often means fewer or smaller instances.

Watch-outs and caveats. Vendor-published, best-case figures.

Levers: Graviton / newer gen · Right-size / autoscale **Sources:** aws.amazon.com/ec2/graviton (customer testimonials)

D Elasticity: Spot, autoscaling and Kubernetes

Four cases on paying only for capacity you use, and on the workload discipline that makes that safe.



Elasticity is the cloud's core promise, but it only pays off when workloads can be moved, interrupted or switched off. These cases cover Spot at scale, Kubernetes node consolidation and simple off-hours schedules.

CS-14

Salesforce Audience Studio

85% of a big-data platform on Spot

Business	Data-management platform (from the Krux acquisition)
Load	~200,000 user queries per second
Infra	~2,500 EMR clusters
Presented	AWS re:Invent 2019

48%

overall savings after moving to Spot

CONTEXT

Audience Studio processes large volumes of third-party audience data in parallel batches, the ideal shape for Spot capacity.

WHAT THEY DID

- ▶ **Spot by default.** More than 85% of the platform's infrastructure runs on Spot Instances, which are spare EC2 capacity sold at a steep discount but reclaimable at short notice.
- ▶ **Designed for interruption.** Jobs are parallel, batch-oriented and restartable, so losing an instance means redoing a slice of work rather than failing a customer request.

Results

- ✓ 48% overall savings across roughly 2,500 daily EMR jobs.

The thinking pattern

Spot is an architecture decision, not a purchasing decision. If a workload is parallel, restartable and flexible about instance type, Spot should be the default and On-Demand the exception.

LESSONS TO STEAL

- Diversify across many instance types and AZs, and use capacity-optimized allocation to reduce interruptions.
- Checkpoint long tasks so an interruption costs minutes, not hours.
- Keep the driver/master nodes On-Demand and run executors on Spot.

Watch-outs and caveats. Spot is a poor fit for stateful, latency-critical or single-instance workloads.

Levers: Spot **Sources:** techtarget.com (re:Invent 2019 coverage)

CS-15

Tinybird

EKS + Karpenter + Spot, and the discipline consolidation demands

Business	Real-time analytics platform
Change	Cluster Autoscaler node groups to Karpenter
Scope	Production and CI/CD on EKS
Published	June 2025

-20%

total AWS bill (up to -90% for CI/CD workloads)

CONTEXT

With Cluster Autoscaler, every combination of instance type, zone and purchase option needed its own node group, so the team was micromanaging node groups and still wasting capacity.

WHAT THEY DID

- ▶ **Adopted Karpenter.** Karpenter launches right-sized nodes directly from pending pods' requirements, choosing from many instance types, including Spot.
- ▶ **Turned on consolidation.** Karpenter continually looks for under-used nodes, moves their pods, and removes or replaces the nodes with cheaper ones, even when pod counts do not change.
- ▶ **Hardened workloads for movement.** Because consolidation evicts pods routinely, services got proper graceful shutdown handling and related safeguards so that moves do not cause errors or micro-outages.

Results

- ✓ AWS bill down about 20% overall and up to 90% for CI/CD; cluster updates and scaling went from days of manual work to automatic.

The thinking pattern

Autoscalers can only save money that workloads allow them to save. Consolidation and Spot both require pods that can be moved at any time, so reliability engineering and cost engineering become the same work.

LESSONS TO STEAL

- Before enabling consolidation: set `terminationGracePeriodSeconds`, handle `SIGTERM`, add readiness probes and `PodDisruptionBudgets`.
- Right-size pod requests first; Karpenter packs requests, not actual usage.
- CI runners are the safest place to start with Spot on Kubernetes.

Watch-outs and caveats. Aggressive consolidation can create churn for stateful or slow-starting pods; use disruption budgets and `do-not-disrupt` annotations.

Levers: Spot · Right-size / autoscale **Sources:**

tinybird.co/blog/how-we-cut-aws-costs-while-scaling-faster-with-eks-karpenter-and-spot-instances

CS-16

Salesforce (EKS fleet to Karpenter)

Single-digit percentages at enormous scale

Business	Enterprise SaaS
Change	Cluster Autoscaler to Karpenter across a very large EKS estate
Primary driver	Faster onboarding, less platform-team toil
Reported	InfoQ, January 2026

~5%

savings in FY2026, with a further 5-10% expected in FY2027

CONTEXT

At Salesforce's scale the migration was motivated mainly by operational agility: faster onboarding and less dependence on central platform teams.

WHAT THEY DID

- **Replaced node-group autoscaling.** Karpenter's real-time instance selection (including Arm, GPU and Spot) and bin-packing replaced Cluster Autoscaler.

Results

- ✓ About 5% cost savings in the first year, with another 5-10% anticipated as bin-packing and Spot usage mature.

The thinking pattern

Cost was a co-benefit of a platform modernization. For very large fleets, a few percent is a large absolute number, and the gains compound as teams adopt the new capabilities.

LESSONS TO STEAL

- Frame platform migrations with both an operational and a cost business case.
- Expect savings to arrive in stages as workloads adopt Spot and tighter requests.

Watch-outs and caveats. Absolute dollar figures were not disclosed.

Levers: Right-size / autoscale · Spot **Sources:** infoq.com/news/2026/01/salesforce-eks-karpenter

CS-17

Moneysmart Group

Spot pools and a 'non-working hours' policy

Business	Personal-finance comparison (Southeast Asia)
Platform	ECS and EKS
Tools	Cost Explorer, Trusted Advisor, Spot.io, Lambda
Source	AWS Startups Blog

~58%

net saving on Spot-managed capacity after vendor fees

CONTEXT

A growing startup analyzed its bill with Cost Explorer and Trusted Advisor and worked with its AWS account team to pick focus areas.

WHAT THEY DID

- **Mixed Spot pools.** A Spot management service ran mixed Spot pools for ECS and EKS with container-driven autoscaling and right-sizing. In one example month they paid about \$2,078 for Spot usage while saving about \$5,656; the vendor took 20% of the savings, leaving roughly 58% net.
- **Switched non-production off at night.** A Lambda function scaled staging EC2 down from 22:00 to 07:00 on weekdays and for the whole weekend.

Results

- ✓ Steadily falling On-Demand share and a large, recurring saving on non-production environments.

The thinking pattern

A week has 168 hours; people work in roughly 50 of them. Non-production that runs around the clock pays for more than two-thirds idle time.

LESSONS TO STEAL

- Running staging only 07:00-22:00 on weekdays (75 of 168 hours) cuts its compute cost by about 55%.
- Compare managed-Spot vendor fees (often a share of savings) with the cost of running Karpenter or Spot Fleet yourself.

Watch-outs and caveats. Scheduled shutdowns need a self-service 'wake up' path, or engineers will simply disable them.

Levers: Spot · Eliminate idle · Right-size / autoscale **Sources:**

aws.amazon.com/blogs/startups/cost-optimization-how-moneysmart-group-leveraged-aws-to-live-and-breathe-efficiency

E Commitments and pricing strategy

How HERE saved \$50M by separating 'what to run' from 'how to pay for it', plus what changed in 2025–2026.



Commitments are the lever that requires no engineering, which is exactly why they should come last: they discount whatever footprint you have, including its waste.

CS-18

HERE Technologies

'Let the builders build', then commit centrally

Business	Mapping and location data
Estate	500+ AWS accounts; 10M+ EC2 instances launched per year
Lever	Programmatic Reserved Instance coverage
Presented	AWS re:Invent 2019

\$50M

saved plus \$150M avoided over ~3.5 years

CONTEXT

HERE deliberately avoided procurement gates that slow engineers down. The result was a sprawling estate that needed a different kind of cost control.

WHAT THEY DID

- ▶ **Programmatic commitments.** HERE adopted Reserved Instances systematically, moving close to 80% of EC2 usage under RIs at roughly 50% off On-Demand rates.
- ▶ **Kept builders unblocked.** Cost control came from central pricing optimization rather than from rigid sourcing processes for teams.

Results

- ✓ About \$50M saved and another \$150M in costs avoided over three and a half years.

The thinking pattern

Separate the question 'what should we run?' (owned by builders) from 'how should we pay for it?' (owned centrally). Commitment management is a portfolio problem: cover the stable baseline, leave the variable top On-Demand or Spot.

LESSONS TO STEAL

- Right-size and remove waste first, then commit to the new, lower baseline.
- Today, Compute Savings Plans offer similar discounts with flexibility across instance families, regions, Fargate and Lambda.
- Database Savings Plans (announced December 2025) bring the same model to RDS, Aurora, DynamoDB, ElastiCache (Valkey) and more: up to 35% off with a one-year, no-upfront commitment.
- Track two KPIs: coverage (share of eligible usage discounted) and utilization (share of commitment actually used).

Watch-outs and caveats. Commitments lock in today's architecture. Over-committing before a Graviton move or a re-architecture can strand discounts.

Levers: Commitments · Culture & ownership Sources: techtarget.com (re:Invent 2019 coverage); aws.amazon.com/about-aws/whats-new/2025/12/database-savings-plans-savings

What changed in 2025–2026 that affects these strategies

- ◆ **Database Savings Plans (Dec 2025).** One-year, no-upfront commitments of \$/hour that discount Aurora, RDS, DynamoDB, ElastiCache (Valkey), DocumentDB, Neptune, Keyspaces, Timestream and DMS by up to 35%, across engines, instance families and regions. Serverless options such as Aurora Serverless v2 and DynamoDB on-demand became discountable for the first time; discounts vary by service (RDS/Aurora instances about 20%, their serverless forms up to 35%).
- ◆ **Resale and pooling rules (2025).** AWS reportedly restricted RIs and Savings Plans from June 2025 to a single end customer's usage, limiting pooled 'group buying' across unrelated customers. Re-check any third-party commitment service you rely on.
- ◆ **Zone-aware routing in ECS Service Connect (July 2026).** On by default; keeps most service-to-service traffic in-zone, cutting cross-AZ charges and latency without code changes.
- ◆ **Exit data transfer (since March 2024).** AWS waives data-transfer-out fees for customers moving data off AWS, on request (37signals had a ~\$250K egress bill comped).

F Architecture, code and network topology

Four cases where the fix was in the design: orchestration granularity, IPC, zone-aware routing and one NAT route.



Some costs are invisible to billing dashboards because they look like ordinary usage. These teams found them by profiling code, tracing bytes and estimating cost per unit of work for each design.

CS-19

Amazon Prime Video

When serverless orchestration costs more than the work

Business	Video streaming (inside Amazon)
Service	Audio/video quality monitoring of live streams
Change	Step Functions + Lambda + S3 to a single-process ECS service
Published	2023

-90%

infrastructure cost for this service

CONTEXT

The Video Quality Analysis team built a tool to inspect every stream for defects such as corruption or audio/video sync problems. The first version used distributed serverless components orchestrated by Step Functions, with S3 as the hand-off point for video frames between components.

WHAT THEY DID

- **Found the per-transition trap.** The workflow performed several state transitions for every second of every stream. That hit account limits, and because Step Functions bills per state transition, cost grew with stream-seconds.
- **Found the per-request trap.** Passing frames between components through S3 generated an enormous number of S3 requests.
- **Collapsed to one process.** All components (media conversion, detectors, orchestration) were packed into a single process running on ECS/EC2, with frames passed in memory. To scale beyond one instance, the service is cloned and each clone runs a subset of detectors, with a lightweight layer distributing requests.

Results

- ✓ Infrastructure cost for the service fell by more than 90%, and it scaled better.

The thinking pattern

Granularity is a cost decision. Serverless and microservices are priced per invocation, transition or request; that is excellent for spiky, low-volume work and ruinous for high-frequency inner loops. The team moved the inner loop into memory and kept distribution only at the coarse level where it paid off.

LESSONS TO STEAL

- Estimate cost per unit of work (per stream-second, per event) for each design before building it.
- Never use object storage or a queue as the data bus for a tight inner loop.
- For high-volume, short workflows, Step Functions Express workflows are billed by requests and duration rather than per transition.
- This is not an argument against serverless in general; it is an argument for choosing granularity per workload.

Watch-outs and caveats. The original article was widely over-interpreted as 'monoliths beat microservices'. The lesson is narrower: match the pricing model to the traffic shape.

Levers: Architecture / code **Sources:** primevideotech.com (original post); devclass.com (May 2023)

CS-20

Recall.ai

How WebSockets over loopback cost \$1M a year

Business	Meeting-bot API (Zoom, Meet, Teams)
Load	Millions of meetings per month; ~1 TB/s of raw video IPC
Fleet	CPU-based video processing on AWS
Published	October 2024

>\$1M

per year saved by replacing an IPC mechanism

CONTEXT

Each bot ran a headless Chromium to join meetings plus a real-time media pipeline, and needed about four CPU cores. The team set a goal to halve that, which would roughly halve the compute bill.

WHAT THEY DID

- ▶ **Profiled before guessing.** CPU profiles showed a surprising hot spot: memory-copy routines, called mostly by the Python WebSocket client receiving data and by Chromium's WebSocket implementation sending it.
- ▶ **Understood the data volume.** Raw decoded video was being streamed out of Chromium's JavaScript environment over a local WebSocket. Uncompressed 1080p at 30 fps is roughly 93 MB/s per stream (1920 x 1080 x 1.5 bytes per frame), so every extra copy was expensive.
- ▶ **Built a better pipe.** After evaluating alternatives such as TCP and Unix domain sockets, the team built a shared-memory ring buffer (including modifying Chromium) so frames could move between processes without repeated copying and framing.

Results

- ✓ Bot CPU usage fell by up to 50%, cutting AWS spend by more than \$1M per year.

The thinking pattern

In a CPU-bound fleet, every percentage point of CPU is a percentage point of the bill. Profiling found a cost that no billing dashboard could have revealed, because it looked like ordinary EC2 usage.

LESSONS TO STEAL

- For compute-dominated bills, a profiler (perf, flame graphs, py-spy) is a cost tool.
- Estimate bytes per second through each hop of a pipeline; high-bandwidth IPC needs zero-copy designs.
- Set a concrete efficiency goal (for example 'half the cores per unit of work') and work backwards.

Watch-outs and caveats. Deep optimizations like forking Chromium carry maintenance cost; they make sense only at large scale.

Levers: Architecture / code **Sources:** recall.ai/blog/how-websockets-cost-us-1m-on-our-aws-bill; infoq.com/news/2024/11/aws-websockets-costs

CS-21

DoorDash

Staying in the zone: cutting cross-AZ data transfer

Business	Food delivery marketplace
Architecture	Cells + microservices on Kubernetes, Envoy service mesh
Change	Zone-aware routing in the mesh
Published	January 2024

Cross-AZ

traffic localized inside each Availability Zone

CONTEXT

Moving from a monolith to cells and microservices improved development speed and resilience, but cross-AZ data transfer costs rose as the number of service-to-service hops grew. On AWS, traffic between AZs is billed in both directions.

WHAT THEY DID

- ▶ **Classified the traffic.** Traffic was split into service-to-service HTTP/gRPC, storage traffic (Aurora PostgreSQL, CockroachDB, Redis, Kafka) and internal infrastructure traffic such as DNS and control-plane chatter.
- ▶ **Tested a hypothesis.** The team suspected HTTP/gRPC calls within a cell were the largest source and prioritized them.
- ▶ **Enabled zone-aware routing.** Their in-house service-mesh control plane was extended to give Envoy each endpoint's AZ, so callers prefer same-zone targets and spill over only when a zone lacks healthy capacity. The cell-based design further localized traffic.

Results

- ✓ Significant reduction in cross-AZ data transfer and fewer network hops, without giving up multi-AZ resilience.

The thinking pattern

Resilience and cost are both properties of topology. Keep traffic local by default, and cross zones only when needed.

LESSONS TO STEAL

- Cross-AZ transfer is charged on send and receive (about \$0.01/GB each way in most regions); microservice fan-out multiplies it.
- Balance capacity per AZ; zone-aware routing on an unbalanced fleet overloads the small zone.
- New in July 2026: Amazon ECS Service Connect has zone-aware routing on by default, keeping over 80% of traffic local when endpoints are balanced. Kubernetes offers topology-aware routing for Services.

Watch-outs and caveats. Zone affinity for stateful systems (Kafka rack awareness, reader endpoints per AZ) needs separate design work.

Levers: Network path / data transfer · Architecture / code Sources: careersatdoordash.com/blog/staying-in-the-zone...; infoq.com/news/2024/01/doordash-service-mesh; aws.amazon.com/blogs/containers/announcing-zone-aware-routing-in-amazon-ecs-service-connect

CS-22

nOps (its own AWS account)

\$12K a month hiding in one NAT route

Business	Cloud cost-management vendor
Symptom	Bandwidth charges up by thousands with no clear cause
Tools	VPC Flow Logs + cost analytics
Published	2025

\$12K

per month removed by fixing a single routing issue

CONTEXT

Even a FinOps company can be surprised by NAT gateway charges, which are billed per hour and per GB processed, and are easy to miss in coarse cost views.

WHAT THEY DID

- ▶ **Traced bytes, not dollars.** VPC Flow Logs showed which sources and destinations were pushing traffic through the NAT gateway.
- ▶ **Fixed the route.** A misconfiguration was sending traffic through NAT that did not need to go there; correcting one route removed the charge.

Results

- ✓ About \$12,000 per month saved.

The thinking pattern

Data-transfer costs are invisible until you attribute bytes to flows. Flow logs turn a mysterious network line item into a list of talkers.

LESSONS TO STEAL

- Add S3 and DynamoDB gateway endpoints everywhere (they are free) and check that every private route table actually has the prefix-list route.
- Add interface endpoints for ECR, CloudWatch, STS and other chatty AWS APIs when volume justifies them.
- Alert on NatGateway-Bytes growth that does not match workload growth.

Watch-outs and caveats. A vendor telling its own story; the pattern, however, is one of the most common in AWS cost reviews.

Levers: Network path / data transfer · Visibility & attribution Sources:

nops.io/reduce-nat-gateway-costs-using-nops-deep-insight-service; vantage.sh/blog/nat-gateway-vpc-endpoint-savings

G Leaving (some of) AWS: repatriation

Six cases at the far end of the spectrum, including one cautionary tale, to calibrate when the cloud premium is worth paying.



Repatriation is the most drastic lever and the most debated. These cases are included not as a recommendation but as calibration: they show which workload shapes and organizational conditions make owned infrastructure cheaper, and what happens when cloud migrations skip modernization.

CS-23

37signals (Basecamp, HEY)

The most public cloud exit

Business	SaaS (Basecamp, HEY and older apps)
Before	\$3.2M/yr cloud run-rate (2022)
Hardware	~\$700K of Dell servers; 18 PB Pure Storage
Timeline	2022-2025

\$10M+

projected savings over five years

CONTEXT

37signals argued it was paying a large premium for elasticity it rarely used: its load was steady and predictable, and it already had data-center space and operations skills.

WHAT THEY DID

- **Moved compute home.** Seven cloud applications, including HEY, were moved from AWS to owned hardware in existing colocation racks. About \$700K of new Dell servers was fully recouped during 2023 as cloud commitments rolled off.
- **Fit inside existing power and racks.** Squeezing new hardware into existing rack and power allowances made savings larger than the original \$7M five-year estimate.
- **Then left S3.** The remaining ~10 PB in S3 (about \$1.5M/yr) is being moved to a dual-data-center Pure Storage array with 18 PB of capacity costing under \$200K per year to run. AWS waived an egress bill of about \$250K under its exit-fee policy.

Results

- ✓ Cloud bill cut from \$3.2M to \$1.3M in 2024 (the remainder was S3).
- ✓ Projected savings of over \$10M across five years, with more capacity and faster machines.

The thinking pattern

Compare the cloud's premium with what it buys you. For steady, predictable workloads run by a team that can operate hardware, elasticity is an insurance policy you may not need.

LESSONS TO STEAL

- Model total cost including people, power, hardware refresh and redundancy, not just server prices.
- Exit costs have fallen: since March 2024 AWS waives data-transfer-out fees for customers moving data off AWS (on request).
- Wait for commitments to expire; stranded RIs and Savings Plans erase early savings.

Watch-outs and caveats. 37signals already had racks, contracts and ops staff. A company without them faces much higher fixed costs.

Levers: Leave the cloud (partially) Sources: world.hey.com/dhh (Oct 2024); datacenterdynamics.com (2024, 2025)

CS-24

Dropbox

Magic Pocket: when storage is the product

Business	Cloud file storage
Change	Most user data from S3 to custom storage in colocation
Completed	Q4 2016
Evidence	2018 S-1 filing

\$74.6M

operating-expense reduction over two years

CONTEXT

Dropbox originally stored file contents in S3 and metadata in its own data centers. As storage became the core of the business, it built its own storage system, Magic Pocket, with custom hardware and software.

WHAT THEY DID

- **Built and migrated.** Over roughly two and a half years, Dropbox built Magic Pocket and moved about 90% of user files onto it, running both systems in parallel during the move.
- **Kept AWS where it helped.** AWS stayed in use for some workloads and regions.

Results

- ✓ In the first year, a \$92.5M drop in third-party data-center spend was offset by \$53M of new depreciation and facilities cost, for a net \$39.5M saving; a further \$35.1M followed the next year, totalling \$74.6M over two years.

The thinking pattern

The CTO summed up the logic in one line: "Nobody is running a cloud business as a charity." At sufficient scale, a commodity you consume heavily is worth building yourself.

LESSONS TO STEAL

- Repatriation makes sense when a workload is huge, steady, specialized, and central to the product.
- Budget for the double-running period during migration; it temporarily raises costs.

Watch-outs and caveats. Very few companies have Dropbox's storage scale or its ability to hire a dedicated infrastructure team.

Levers: Leave the cloud (partially) **Sources:** sec.gov (Dropbox S-1, 2018); datacenterknowledge.com

CS-25

X (formerly Twitter)

Blob storage out, data processing slimmed down

Business	Social network
Claim date	October 2023
Change	Media/blob artifacts moved on-prem; cloud processing optimized
Evidence	Company engineering post

-60%

monthly cloud cost (company claim)

CONTEXT

Under severe cost pressure and amid publicized disputes with cloud vendors, X's engineering team reported a year of infrastructure changes.

WHAT THEY DID

- **Moved blobs home.** All media and blob artifacts were moved out of the cloud, cutting cloud storage size by about 60%.
- **Cut processing.** Cloud data-processing costs were reported to be down about 75%.

Results

- ✓ Monthly cloud cost reportedly down 60%.

The thinking pattern

Blob storage and heavy data processing are the two cloud line items with the largest gap between rental and ownership cost.

LESSONS TO STEAL

- If you already own data centers, moving bulk storage back is often the biggest single lever.
- Separate claims from verified outcomes; public statements rarely include total cost of ownership.

Watch-outs and caveats. Figures are company claims made during a period of vendor disputes and deep staff cuts; they are not independently verified.

Levers: Leave the cloud (partially) · Storage tiering Sources: datacenterdynamics.com (Nov 2023); theregister.com (Oct 2023)

CS-26

Prerender.io

A data-transfer-heavy workload that outgrew AWS pricing

Business	Pre-rendering JavaScript pages for crawlers
Load	~70,000 pages per minute; ~560M cached pages
Change	AWS to in-house infrastructure
Completed	Late 2022

\$1M to \$200K
annual server cost (-80%)

CONTEXT

Prerender caches and serves rendered HTML. Its costs were dominated by servers, storage and data transfer, all of which grow with customer traffic.

WHAT THEY DID

- ▶ **Planned carefully.** Because downtime would hurt customers' search rankings, the team planned the migration in stages.
- ▶ **Moved the heavy parts.** Caching, storage and rendering were moved to owned infrastructure over a little more than three months.

Results

- ✓ Annual costs fell from a projected \$1M+ to about \$200K, saving roughly \$800K per year.

The thinking pattern

Workloads whose bill is mostly bandwidth and storage, with little need for managed services, get the least value from cloud premiums.

LESSONS TO STEAL

- Look at what share of your bill is egress and raw storage; those are where alternatives are cheapest.
- Migrate in stages with rollback paths; SEO-sensitive services cannot tolerate long outages.

Watch-outs and caveats. Savings figures come from the company's own account.

Levers: Leave the cloud (partially) · Network path / data transfer Sources: levelup.gitconnected.com (interview with Prerender tech lead); indiehackers.com

CS-27

OneUptime

A 28-node Kubernetes cluster moved to bare metal

Business	Open-source observability/status pages
Before	\$456K+/yr for a 28-node AWS Kubernetes cluster
After	Colocated bare metal, MicroK8s, Ceph
Published	2023, with a two-year follow-up

\$230K

saved per year (55%+)

CONTEXT

Because the product already ran on Kubernetes with Helm, the cluster was portable. Modern tooling (Docker, Kubernetes, Ceph) made running on bare metal far easier than it had been a decade earlier.

WHAT THEY DID

- **Phased cutover.** MicroK8s on bare metal was tested with a subset of traffic, more workloads were shifted over weeks while being monitored, and all traffic was routed over once stable.
- **Answered the critics.** A two-year follow-up addressed common objections: whether Reserved Instances would have closed the gap, how to fail over a single rack, and the people cost.

Results

- ✓ Over \$230K saved per year, sustained over two years.

The thinking pattern

Kubernetes is a portability layer; if you already pay the complexity cost of running it, the cost of moving the underlying machines drops sharply.

LESSONS TO STEAL

- Compare against the best cloud price (with Savings Plans and Spot), not the On-Demand list price.
- Plan for rack-level and site-level failure explicitly when leaving multi-AZ cloud.

Watch-outs and caveats. A small company with strong infrastructure skills; results depend heavily on in-house expertise.

Levers: Leave the cloud (partially) **Sources:** oneuptime.com/blog/post/2023-10-30-moving-from-aws-to-bare-metal

CS-28

GEICO (cautionary tale)

A decade of lift-and-shift, then a costly reversal

Business	US auto insurer
Cloud footprint	Multiple providers; ~\$300M annual cloud bill
Change	Repatriating to OCP-based private cloud (OpenStack, Kubernetes)
Reported	2024-2025

2.5x

growth in cloud bills after ten years of migration

CONTEXT

GEICO moved to public cloud to cut cost and complexity, largely by lifting and shifting legacy applications across several providers.

WHAT THEY DID

- **Diagnosed the failure.** Ten years in, not everything had migrated, bills had risen about 2.5 times, and reliability problems had increased. Running legacy applications unchanged in the cloud proved very expensive; storage was singled out as one of the costliest things to do in the cloud.
- **Rebuilt on open hardware.** GEICO is building private cloud capacity on Open Compute Project hardware with open-source software, repatriating many workloads.

Results

- ✓ GEICO reports more than 50% lower unit cost per compute core and more than 60% lower unit cost per GB of storage compared with its public-cloud and previous OEM costs.

The thinking pattern

Lift-and-shift preserves on-prem inefficiencies and adds cloud prices on top. The cloud rewards applications re-designed for elasticity; it punishes static, always-on legacy systems.

LESSONS TO STEAL

- Do not migrate without a modernization plan; right-size and re-architect as you move.
- Spreading workloads across many clouds without a clear reason multiplies complexity and cost.
- Storage-heavy, compliance-driven retention is a prime candidate for tiering or on-prem.

Watch-outs and caveats. GEICO's situation spans several cloud providers, not only AWS; its unit-cost figures are self-reported.

Levers: Leave the cloud (partially) · Storage tiering Sources: thenewstack.io; thestack.technology (interviews with Rebecca Weekly); infoworld.com

04 Cross-case patterns

Fourteen thinking patterns that recur across the case studies, with the evidence behind each and how to apply it.



Techniques change as AWS launches new products; thinking patterns last. The fourteen below are the recurring moves that separated the successful programmes from generic cost-cutting. Each lists the cases that demonstrate it and a concrete way to apply it.

P01 Start from the bill, not the diagram

Engineers tend to optimize the components they know. The bill shows what actually costs money, which is often something nobody thinks about (a CDN, backups, a NAT gateway).

Evidence: Levels.fyi (1, 2), Segment, Duolingo, Lyft **Apply it:** Rank the top 20 line items by cost and by month-over-month growth; investigate both lists.

P02 Ask 'expected or buggy?' and watch slopes

A large number can be fine; a small number growing every day is a lead. Levels.fyi chased a \$26 line item because it kept rising; Segment noticed the bill rising faster than usage.

Evidence: Levels.fyi (2), Segment **Apply it:** Plot each service's daily cost with a 30-day trend line; alert when growth outpaces your unit metric.

P03 Treat defaults and old decisions as suspects

Almost nothing in these cases was a bug. Log destinations, retention periods, autoscaling rules and capacity bumped during incidents were each correct once and expensive later.

Evidence: Levels.fyi (2), Segment, Duolingo **Apply it:** For every large resource, find the decision behind it and ask whether its reason still holds.

P04 Price every copy and every day kept

Storage cost is bytes x copies x duration. Backups, versions, snapshots and logs multiply silently.

Evidence: Levels.fyi (2), Duolingo, Canva **Apply it:** Write retention as a table (what, how many copies, how long, why) and enforce it in code.

P05 Estimate cost per unit of work for each design

Services priced per request, transition or GB processed turn traffic patterns into bills. High-frequency inner loops belong in memory, not in queues, object stores or orchestrators.

Evidence: Prime Video, Levels.fyi (2), Duolingo, DoorDash **Apply it:** In design reviews, compute cost per request/event/stream-second for each proposed path.

P06 Normalize by a business unit

Absolute spend hides efficiency. Cost per ride, per event or per GB reveals it and makes teams comparable.

Evidence: Lyft, Segment, Canva **Apply it:** Publish one unit-cost number per product weekly, with a team breakdown.

P07 Do the payback maths before flipping a switch

Migrations, transitions and hardware purchases have one-off costs. The winning teams ranked options by payback period.

Evidence: Canva, Rewind, 37signals, Dropbox **Apply it:** Payback months = one-off cost / monthly saving. Prioritize anything under six months.

P08 Verify from the inside before deleting

Metrics on a resource do not prove it is used; its own internals do (Redis INFO, connection counts, access logs).

Evidence: Levels.fyi (1), Duolingo **Apply it:** Snapshot, inspect, scale to zero or block access for a week, then delete.

P09 Change the unit price with low-risk swaps

Graviton, gp3, t4g and newer generations offer lower prices for equal or better performance with little code change.

Evidence: Pinterest, Honeycomb, Zomato, Zendesk, Levels.fyi (1) **Apply it:** Build multi-arch images once; migrate the largest compute consumer first.

P10 Make workloads movable, then let automation move them

Spot, consolidation and schedules only save money when workloads handle interruption gracefully.

Evidence: Salesforce, Tinybird, Moneysmart **Apply it:** Graceful shutdown, disruption budgets and checkpointing before enabling Spot or consolidation.

P11 Optimize first, commit last

Commitments discount whatever you run, waste included, for one to three years.

Evidence: HERE, Airbnb, Duolingo **Apply it:** Commit only to the trough of the optimized baseline; ladder purchases quarterly.

P12 Profile when compute dominates

In CPU-bound fleets, inefficiency in code looks like ordinary EC2 usage on the bill. Only a profiler finds it.

Evidence: Recall.ai, Segment **Apply it:** Set a 'cores per unit of work' goal and profile the hottest service against it.

P13 Measure both sides of a change, and distrust fresh billing data

Cost Explorer's latest days are incomplete, and a change can move cost somewhere else (backend load, another AZ).

Evidence: Levels.fyi (2) **Apply it:** Use a control line item and real-time signals; wait several days before declaring victory.

P14 Know what the cloud premium is buying

Elasticity, managed operations and speed are worth paying for; for steady, storage-heavy loads with in-house skills, they may not be. Lift-and-shift gets the worst of both worlds.

Evidence: 37signals, Dropbox, Prerender, OneUptime, GEICO **Apply it:** Score each large workload on variability, managed-service dependence, storage/egress share and ops capability.

Lever matrix: which case used what

A dot means the lever was a material part of that case. Read down a column to find every example of a lever; read across a row to see how many levers a single programme combined. Legend: ELIM eliminate idle, SIZE right-size or autoscale, STOR storage tiering, GRAV Graviton / newer generation, SPOT Spot capacity, COMM commitments, RET retention and logging, NET network path and data transfer, ARCH architecture or code, VIS visibility and attribution, CULT culture and ownership, EXIT leaving the cloud (partially).

Case	ELIM	SIZE	STOR	GRAV	SPOT	COMM	RET	NET	ARCH	VIS	CULT	EXIT
CS-01 Levels.fyi, Part 1	●	●	●	●		●	●					
CS-02 Levels.fyi, Part 2							●	●	●	●		
CS-03 Segment	●	●							●	●		
CS-04 Duolingo	●	●				●	●		●	●	●	
CS-05 Lyft					●				●	●	●	
CS-06 Airbnb			●			●				●	●	

Case	ELIM	SIZE	STOR	GRAV	SPOT	COMM	RET	NET	ARCH	VIS	CULT	EXIT
CS-07 Canva			●							●		
CS-08 Snap Inc.			●									
CS-09 Rewind			●									
CS-10 Pinterest		●	●	●								
CS-11 Honeycomb				●								
CS-12 Zomato				●								
CS-13 Zendesk		●		●								
CS-14 Salesforce Audience Studio					●							
CS-15 Tinybird		●			●							
CS-16 Salesforce		●			●							
CS-17 Moneysmart Group	●	●			●							
CS-18 HERE Technologies						●					●	
CS-19 Amazon Prime Video									●			
CS-20 Recall.ai									●			
CS-21 DoorDash								●	●			
CS-22 nOps								●		●		
CS-23 37signals												●
CS-24 Dropbox												●
CS-25 X			●									●
CS-26 Prerender.io								●				●
CS-27 OneUptime												●
CS-28 GEICO			●									●

05 Technique catalogue by AWS service

Every technique from the case studies, organized by the service it applies to, with watch-outs and the cases that used it.



Use this section as a checklist during a cost review. Prices mentioned are approximate us-east-1 list prices at the time of writing; always confirm on the current AWS pricing pages.

Compute: EC2

Technique	Why it saves	Watch-outs	Seen in
Delete idle and temporary instances	Stops paying for forgotten capacity and attached EBS.	Verify ownership; stopped instances still pay for EBS and Elastic IPs.	Levels.fyi (1), Duolingo
Right-size (Compute Optimizer)	Most fleets run far below capacity.	Check memory as well as CPU; watch p99 latency.	Segment, Duolingo
Graviton / newer generation	Around 20% lower price per instance and often better performance.	Native dependencies, x86-only binaries, licensed software.	Pinterest, Honeycomb, Zomato, Zendesk
Spot for tolerant workloads	Up to ~90% off On-Demand.	Interruptions; diversify types and AZs; checkpoint.	Salesforce, Lyft CI, Moneysmart, Tinybird
Bin-pack, fewer instance types	Higher utilization; simpler commitments.	Noisy neighbours; blast radius.	Segment
Schedule non-production	Pays only for working hours.	Provide a self-service wake-up.	Moneysmart

Compute: containers and serverless

Technique	Why it saves	Watch-outs	Seen in
ECS/Fargate task right-sizing	Tasks are billed on requested vCPU/memory, not usage.	Duolingo found 90-95% memory utilization safe; test your own.	Duolingo, Levels.fyi (1)
Lower minimum tasks, calmer scale-out	Fewer idle tasks off-peak.	Keep headroom for cold starts.	Duolingo, Levels.fyi (1)
EKS: Karpenter + consolidation	Just-in-time nodes, continuous repacking, Spot mix.	Requires graceful shutdown and disruption budgets.	Tinybird, Salesforce
EKS: right-size pod requests first	Schedulers pack requests, not usage.	Use VPA recommendations; avoid OOM.	Tinybird
EKS: stay on standard support	Extended-support versions carry a much higher control-plane fee (about \$0.60 vs \$0.10 per cluster-hour).	Plan upgrades on a cadence.	General practice
Lambda on arm64	About 20% lower duration price, often faster.	Check native modules and layers.	Levels.fyi (1)
Step Functions: avoid per-second transitions	Standard workflows bill per state transition.	Use Express workflows or in-process orchestration for high volume.	Prime Video

Storage: S3, EBS, backups, registries

Technique	Why it saves	Watch-outs	Seen in
S3 lifecycle to IA / Glacier IR / Deep Archive	Standard ~\$0.023, IA ~\$0.0125, Glacier IR ~\$0.004, Deep Archive ~\$0.00099 per GB-month.	Per-object transition fees; 30/90/180-day minimums; 128 KB minimum billable size.	Canva, Snap, Rewind, Levels.fyi (1), Pinterest
Measure access first	Storage Class Analysis and Storage Lens show real access patterns.	Product changes can reheat cold data.	Canva
Expire noncurrent versions	Versioned buckets keep every overwrite forever by default.	Keep enough history for recovery.	Duolingo
Abort incomplete multipart uploads	Orphaned parts are billed but invisible in listings.	Add a lifecycle rule to every bucket.	General practice
Intelligent-Tiering	Automatic tiering for unpredictable access.	Per-object monitoring fee; objects under 128 KB are not tiered.	Airbnb
Pause event consumers during bulk transitions	Prevents a flood of storage-class events.	Re-attach listeners afterwards.	Levels.fyi (1)
EBS gp2 to gp3	About 20% cheaper per GB; IOPS and throughput decoupled from size.	Set IOPS/throughput to match gp2 burst if needed.	Levels.fyi (1)
Delete unattached volumes and old snapshots	Orphans accumulate after instance deletion.	Snapshot before deleting.	Segment, Levels.fyi (1)
Tiered backup retention (GFS)	Keeps recovery points while cutting copies.	Document the policy for auditors.	Levels.fyi (2)
ECR lifecycle policies	Old images pile up per build.	Keep images referenced by running or rollback releases.	Levels.fyi (1)

Databases and caches

Technique	Why it saves	Watch-outs	Seen in
Graviton classes and gp3 storage for RDS	Lower price per vCPU and per GB.	Test extensions and drivers.	Levels.fyi (1)
Aurora I/O-Optimized	Removes per-I/O charges for I/O-heavy clusters.	Evaluate when I/O is roughly a quarter or more of Aurora spend.	Duolingo
DynamoDB key design	Capacity is sized by the hottest partition.	Needs data-model changes.	Segment
DynamoDB TTL	Automatic deletion of stale items.	Does not clean existing rows unless you backfill the attribute.	Duolingo
DynamoDB Standard-IA table class	Cheaper storage for large, rarely read tables.	Higher read/write prices.	Duolingo
Capacity mode review	On-demand for spiky, provisioned + autoscaling for steady.	Revisit as traffic matures.	Lyft (top-25 tables)
Find orphaned caches and databases	Legacy features leave them behind.	Check connections from inside first.	Levels.fyi (1), Duolingo
Valkey engine for ElastiCache	Valkey is priced below Redis OSS on ElastiCache.	Validate client compatibility.	Levels.fyi (2) used Valkey
Database Savings Plans	Up to 35% off with a one-year, no-upfront commitment, across engines and regions.	Commit after right-sizing.	Chapter E

Networking and data transfer

Technique	Why it saves	Watch-outs	Seen in
S3 and DynamoDB gateway endpoints	Free; removes NAT processing (~\$0.045/GB) for that traffic.	Confirm every private route table has the prefix-list route.	nOps, general practice
Interface endpoints for chatty AWS APIs	Cheaper than NAT for ECR pulls, CloudWatch, STS at volume.	Hourly fee per AZ; do the maths.	nOps
Zone-aware routing	Avoids cross-AZ charges (~\$0.01/GB each way).	Balance capacity per AZ.	DoorDash
Remove hairpins through CDN/WAF	Stops paying per-request edge prices for internal traffic.	Replace edge caching first (shadow-run).	Levels.fyi (2)
PrivateLink for overlapping CIDRs	Private connectivity without re-addressing networks.	Check AZ coverage of endpoints.	Levels.fyi (2)
Trace bytes with VPC Flow Logs	Turns a network line item into a list of talkers.	Flow logs have their own cost; sample.	nOps
Release unused public IPv4 addresses	AWS charges for every public IPv4 address (about \$0.005/hour each).	Prefer private subnets + endpoints.	General practice
Cheaper CDN or better cache-hit ratio	CDN can be the largest line item for content sites.	Egress and request pricing differ widely.	Levels.fyi (1)

Observability

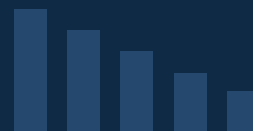
Technique	Why it saves	Watch-outs	Seen in
Set log retention everywhere	CloudWatch log groups default to never expire.	Keep audit logs per policy.	Levels.fyi (1), Duolingo
Stop logging payloads and stack traces	Ingestion is priced per GB (around \$0.50 in the standard class).	Keep error-level detail; sample the rest.	Levels.fyi (2), Duolingo
Cheaper destinations for bulk logs	S3 (or the CloudWatch Infrequent Access class) is far cheaper for rarely read logs.	Lose some real-time query features.	Levels.fyi (2)
Filter WAF/CDN logs	Keep blocked or suspicious requests, not every allowed one.	Security team sign-off.	Levels.fyi (2)
Automated anomaly detection	Catches regressions within days.	Tune thresholds per service.	Duolingo

Pricing and commitments

Technique	Why it saves	Watch-outs	Seen in
Compute Savings Plans	Flexible across families, regions, Fargate, Lambda.	Commit to the trough, not the average.	Airbnb
Reserved Instances at high coverage	About 50% off for predictable baselines.	Less flexible than SPs.	HERE
Database Savings Plans	Extends commitment discounts to databases, including serverless.	One-year term only.	Chapter E
Managed-Spot or group-buying services	Outsourced optimization for a share of savings.	Fees (e.g. 20% of savings); resale rules.	Moneysmart, Levels.fyi (1)

06 Maths and decision tools

The back-of-envelope calculations the case-study teams used, turned into reusable formulas, tables and checklists.



6.1 Payback period

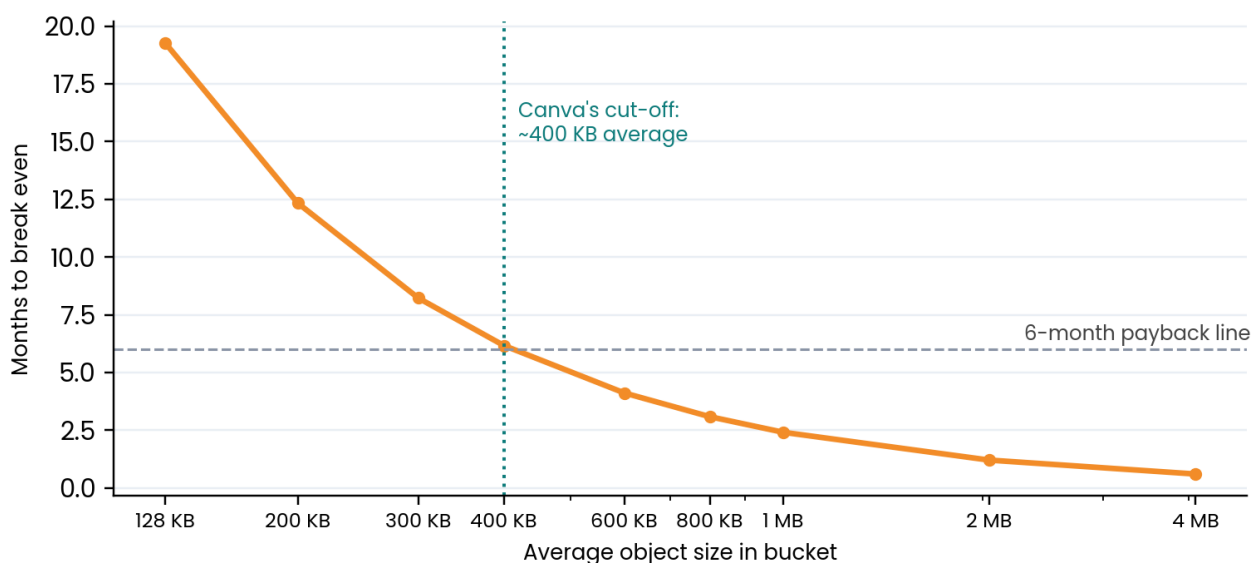
$\text{payback_months} = \text{one_off_cost} / \text{monthly_saving}$

Canva: \$1.6M / \$0.3M per month = about 5.3 months. Rewind: \$103K / \$16K per month = about 6.4 months. 37signals: roughly \$700K of hardware against about \$160K per month of reduced cloud spend, recouped within 2023. Anything under six months is usually an easy yes; above eighteen months needs a strategic reason.

6.2 S3 transition break-even by object size

Transitions are billed per object and savings accrue per byte, so average object size decides payback. Using approximate list prices (Standard-IA \$0.0125, Glacier IR \$0.004 per GB-month, about \$0.02 per 1,000 transition requests):

$\text{months} = (0.02 / 1000) / (\text{object_GB} \times (0.0125 - 0.004))$



Average object size	Months to break even	Verdict
128 KB	19.3	Only if data will stay cold for years
200 KB	12.3	Marginal
400 KB	6.2	Canva's cut-off; good
1 MB	2.4	Excellent
4 MB	0.6	Immediate win

Glacier IR also bills a minimum of 90 days per object and 128 KB per object, and its retrieval charges are higher than Standard-IA's. Include expected retrieval volume if data is not truly cold.

6.3 Scheduling non-production

Schedule	Hours per week	Compute saving vs 24x7
24 x 7	168	0%
Weekdays 07:00-22:00 (Moneysmart)	75	55%
Weekdays 08:00-20:00	60	64%
Weekdays 09:00-19:00	50	70%

6.4 NAT gateway versus gateway endpoint

A workload in private subnets pushing 10 TB a month to S3 through a NAT gateway pays about 10,240 GB x \$0.045 = \$461 per month in processing, plus about \$33 per NAT gateway per month in hourly charges (times the number of AZs). Through an S3 gateway endpoint the same traffic costs nothing extra. For ECR image pulls, CloudWatch or STS, interface endpoints cost about \$0.01 per hour per AZ plus \$0.01 per GB, which beats NAT once volume is more than a few hundred GB a month.

6.5 The cost of one log line

A Lambda invoked 50 million times a month that logs a 20 KB event on every call writes roughly 950 GB a month. At around \$0.50 per GB ingested, that is about \$475 per month before storage, and storage keeps growing if the log group never expires. A 200-byte summary line instead writes about 9.5 GB, roughly \$5 a month: the same 99% reduction Levels.fyi saw.

6.6 Cross-AZ chatter

Two services exchanging 50 TB a month across AZs pay about \$0.01 per GB in each direction, around \$1,000 a month. If zone-aware routing keeps 80% of that traffic local, the charge falls to roughly \$200. The saving grows linearly with every new hop in a microservice call chain.

6.7 Sanity-checking a claimed saving

Levels.fyi cut RDS snapshot storage from 15.7 TB to 3 TB and reported about \$1,200 a month in savings. At roughly \$0.095 per GB-month for RDS backup storage beyond the free allowance, 12.7 TB is about \$1,235 a month, which matches. Reproducing a claimed saving from list prices is a quick way to check that you understand where the money went.

6.8 Commitment sizing

- Finish right-sizing, Graviton moves and scheduling first; commitments freeze the footprint.
- Look at hourly eligible spend over the last 30-60 days and commit to roughly its trough, not its average.
- Ladder purchases (for example quarterly) so commitments expire at different times and can track change.
- Track coverage (discounted share of eligible usage) and utilization (share of commitment used); aim high on utilization before pushing coverage.
- Prefer Compute Savings Plans and Database Savings Plans when your instance families or engines are still changing.

6.9 Checklists

Is it a Spot candidate?

- ☐ Stateless or checkpointed
- ☐ Tolerates a two-minute termination notice
- ☐ Runs on several instance types
- ☐ Horizontally scaled
- ☐ Retries are cheap

Is it Graviton-ready?

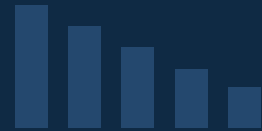
- ☐ Interpreted or JVM runtime, or you control the build
- ☐ No x86-only binaries or licensed agents
- ☐ Base images published for arm64
- ☐ CI can build multi-arch images
- ☐ Benchmark on production-like load

Would leaving pay off?

- ☐ Load is steady and predictable
- ☐ Storage and egress dominate the bill
- ☐ Few managed-service dependencies
- ☐ In-house ops and hardware skills
- ☐ Existing racks, power, contracts

07 Pitfalls and anti-patterns

Twenty mistakes the case-study teams made or narrowly avoided, each with a guard rail you can adopt.

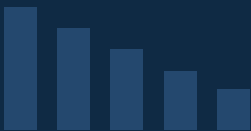


Pitfall	Seen in	Guard rail
Committing before optimizing	HERE (context), general	Right-size and modernize first; commit to the new trough.
Trusting the last few days in Cost Explorer	Levels.fyi (2)	Check a control line item; wait several days.
Deleting based on activity graphs	Levels.fyi (1)	Inspect from inside (INFO, connections); snapshot first.
Bulk storage-class changes firing events	Levels.fyi (1)	Pause or filter S3 event consumers during transitions.
Ignoring transition fees and minimum durations	Canva, Rewind	Compute payback per bucket; align ages with minimums.
Assuming a new TTL cleans old data	Duolingo	Backfill the TTL attribute or run a one-off purge.
Never reverting incident scale-ups	Segment	Create a scale-down ticket in every incident review.
Staging larger than production	Duolingo	Budget alerts per environment; schedules for non-prod.
Per-transition orchestration in hot loops	Prime Video	Estimate cost per unit of work; keep inner loops in memory.
Internal traffic hairpinning through the edge	Levels.fyi (2)	Private paths (PrivateLink, endpoints) for service calls.
Gateway endpoint without the route	nOps	Audit route tables for the S3/DynamoDB prefix-list route.
Round-robin across AZs by default	DoorDash	Zone-aware or topology-aware routing with balanced capacity.
Spot or consolidation without graceful shutdown	Tinybird	SIGTERM handling, PDBs, readiness probes first.
Single-instance proofs of concept	Levels.fyi (2)	Test across every AZ and instance group before cutover.
Timeouts that measure the wrong thing	Levels.fyi (2)	Measure on an idle path; do not discard completed work.
Lift-and-shift without modernization	GEICO	Right-size and re-architect during migration.
Comparing exits to On-Demand list prices	OneUptime (critics)	Compare with the best cloud price incl. SP and Spot.
Maximal compliance retention	Levels.fyi (2)	Define, document and follow a tiered retention policy.
Logging full payloads and stack traces	Levels.fyi (2), Duolingo	Summary lines, sampling, error-only in loops.
Forgetting double-running costs	Dropbox	Budget for parallel systems during migrations.

08

A 90-day playbook

A sequenced plan that combines the case-study methods, plus the rhythm that keeps savings from eroding.



Days 1-7: See

- ☐ Enable CUR 2.0 / Data Exports to S3 and query with Athena (or load into your warehouse).
- ☐ Agree a tag schema (owner, service, environment, cost-centre) and enforce it with tag policies and IaC checks.
- ☐ Turn on Cost Anomaly Detection and per-team budgets.
- ☐ Create a read-only analysis role; optionally let an LLM agent use it to answer precise questions (Levels.fyi method).
- ☐ Pick one unit-cost metric and publish its baseline.

Weeks 2-4: Stop and shrink

- ☐ Delete idle instances, unattached EBS, old snapshots, unused load balancers, Elastic IPs and public IPv4 addresses.
- ☐ Set CloudWatch log retention everywhere; remove payload and stack-trace logging; filter WAF/CDN logs.
- ☐ Add S3 lifecycle rules: noncurrent versions, incomplete multipart uploads, cold-data transitions.
- ☐ Add S3/DynamoDB gateway endpoints; migrate gp2 to gp3; add ECR lifecycle policies.
- ☐ Schedule non-production to working hours.

Weeks 4-8: Swap and flex

- ☐ Right-size with Compute Optimizer and container request data; lower minimum task counts.
- ☐ Run a Graviton pilot on the largest compute consumer; build multi-arch images.
- ☐ Evaluate Aurora I/O-Optimized, DynamoDB capacity modes and table classes.
- ☐ Move CI/CD and batch to Spot; adopt Karpenter with consolidation where workloads are ready.

Weeks 8-12: Redesign and commit

- ☐ Fix the top three chatty paths: hairpins, cross-AZ fan-out, per-request inner loops.
- ☐ Run storage-class migrations ranked by payback period.
- ☐ Buy Compute and Database Savings Plans on the new baseline, laddered.
- ☐ Write a one-page retention policy and encode it in Terraform modules.

Keeping the savings: the operating rhythm

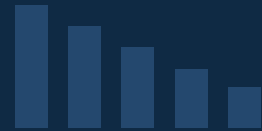
Cadence	Ritual
Weekly	Anomaly review; top movers by service and team; check the unit-cost trend.
Monthly	Showback per team; review waste list; celebrate wins publicly (Lyft-style leaderboards).
Quarterly	Commitment coverage and utilization; architecture review of top five line items; retention audit.
Every design doc	A cost estimate per unit of work and a retention section (Duolingo practice).

KPIs worth tracking

KPI	Definition	Healthy target
Unit cost	Spend / business unit (ride, event, active user, GB)	Falling quarter over quarter
Commitment utilization	Used commitment / purchased commitment	Above 95%
Commitment coverage	Discounted eligible usage / eligible usage	70-85% of steady baseline
Tag coverage	Spend with owner and service tags / total	Above 95%
Waste ratio	Idle + orphaned + unattached spend / total	Below 5%
Time to detect anomalies	Days from cost jump to owner notified	Under 2 days
Graviton and Spot share	Share of eligible compute on Graviton / Spot	Rising

09 Applying this to your stack

Concrete checks for a DevSecOps team running multi-cloud Kubernetes, Vault, GitOps, PostgreSQL and SQL Server workloads.



The patterns above translate directly to a platform estate built on EKS (alongside OKE and GKE), HashiCorp Vault, Argo CD, Terraform, PostgreSQL with pgBackRest on S3, SQL Server Always On availability groups on AWS and GitHub Enterprise CI. The checks below are ordered roughly by effort.

Kubernetes (EKS first; OKE and GKE have equivalents)

- ▶ Right-size pod requests from real usage (VPA recommender or Goldilocks) before touching node autoscaling.
- ▶ Adopt Karpenter with consolidation for stateless workloads and CI runners; add PodDisruptionBudgets and SIGTERM handling first (Tinybird lesson).
- ▶ Create Graviton node pools and build multi-arch images in CI (Honeycomb lesson).
- ▶ Keep every cluster on a standard-support Kubernetes version to avoid extended-support control-plane fees, and count clusters: each carries a fixed hourly fee.
- ▶ Enable topology-aware routing for chatty Services; add ECR and S3 endpoints so image pulls skip NAT.
- ▶ On GKE and OKE, the same ideas apply through Spot VMs, preemptible instances and flexible shapes.

PostgreSQL backups with pgBackRest on S3

- ▶ Apply the Levels.fyi lesson: write down the retention you actually need (full, differential and WAL retention) and configure pgBackRest expiry to match it, rather than keeping everything.
- ▶ If the repository bucket has versioning enabled, expire noncurrent versions and abort incomplete multipart uploads; otherwise expired backups keep costing money.
- ▶ Route WAL archiving and backup traffic from private subnets through an S3 gateway endpoint to avoid NAT processing fees.
- ▶ Be careful with lifecycle transitions on the repository prefix: objects that pgBackRest expires early may incur minimum-duration charges in IA or Glacier classes. Do the payback maths first.
- ▶ Use compression in pgBackRest and keep a cross-region copy only if your DR requirement actually demands it.

SQL Server Always On on AWS

- ▶ Licensing often costs more than the compute. Re-check edition (Standard vs Enterprise), vCPU counts and whether license-included or bring-your-own licensing is cheaper for each node.
- ▶ For BYOL, the EC2 'Optimize CPUs' option can reduce active cores (and per-core licences) on memory-heavy instances.
- ▶ Confirm the licensing position of passive secondary replicas with your Microsoft agreement; rules vary.
- ▶ Measure cross-AZ replication traffic; synchronous replicas across AZs are billed for data transfer in both directions.
- ▶ Compare gp3 with provisioned IOPS against io2 for data and log volumes.

Vault, observability and GitOps

- ▶ Right-size Vault nodes (often over-provisioned) and run them on gp3; Vault ships arm64 builds for Graviton.
- ▶ Route high-volume audit logs to S3 with lifecycle rules instead of premium log ingestion, keeping what compliance requires.
- ▶ Encode cost guard rails in Terraform modules: default gp3, mandatory tags, S3 lifecycle rules, log retention, no public IPv4 by default. Enforce with policy-as-code in CI.
- ▶ Show a cost diff on infrastructure pull requests so reviewers see the price of a change.
- ▶ Use Argo CD to reconcile scheduled scale-down for non-production namespaces.

CI/CD and GitHub Enterprise runners

- ▶ Run self-hosted runners on Spot and arm64 with autoscaling runner sets; CI is the safest Spot workload (Lyft, Tinybird).
- ▶ Set artifact and cache retention; cache container layers to reduce repeated NAT egress for base images.

LLM-assisted cost forensics

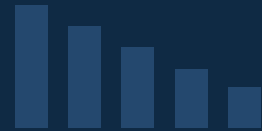
- ▶ Create a dedicated read-only role (Cost Explorer, CUR via Athena, CloudWatch read) with no write permissions, and block writes with an SCP as a second layer.
- ▶ Log every call with CloudTrail, and keep CUR data out of prompts sent to external services unless policy allows it.
- ▶ Ask measurable questions: which log groups grew fastest last month and who writes to them; which line items grew every day for 30 days; which NAT gateways moved the most bytes and to where.

Where security and cost overlap

Several savings in this report also shrink attack surface: private paths instead of public edges (Levels.fyi's PrivateLink move), fewer public IPv4 addresses, filtered WAF logs that make real attacks easier to see, and deleting forgotten instances and caches that nobody patches. Framing cost work as hygiene makes it easier to win support from both security and finance.

10 Sources and method

Where every number came from, and how to read them.



Method

Cases were selected if they were public, AWS-centric (or directly about leaving AWS), and included concrete numbers or specific techniques. Primary sources (company engineering blogs, SEC filings, AWS case studies, conference talks) were preferred; where only secondary coverage existed, that is noted in the case's caveats. All descriptions are paraphrased summaries. Savings are as reported by the companies or vendors and were not independently audited; percentages often refer to a single workload rather than a whole bill. Prices are approximate us-east-1 list prices at the time of writing (September 2026) and change frequently.

References

- [1] **Levels.fyi, Part 1**
<https://www.levels.fyi/blog/slashing-aws-bill.html>
- [2] **Levels.fyi, Part 2**
<https://www.levels.fyi/blog/slashing-our-aws-bill-at-levelsfyi-part-2.html>
- [3] **Segment**
<https://segment.com/blog/the-million-dollar-eng-problem>
- [4] **Duolingo**
<https://blog.duolingo.com/reducing-cloud-spending/>
- [5] **Duolingo FinOps**
<https://blog.duolingo.com/finops>
- [6] **Lyft, HERE, Salesforce (re:Invent 2019)**
<https://www.techtarget.com/it-infrastructure/news/252475058/New-AWS-cost-management-tool-instance-tactics-to-cut-cloud-bills>
- [7] **Airbnb**
<https://aws.amazon.com/solutions/case-studies/airbnb-optimizes-usage-and-costs-case-study/>
- [8] **Canva**
<https://aws.amazon.com/blogs/storage/how-canva-saves-over-3-million-annually-in-amazon-s3-costs>
- [9] **Snap**
<https://aws.amazon.com/solutions/case-studies/snap-case-study/>
- [10] **Rewind**
<https://rewind.com/blog/saving-money-aws-glacier-instant-retrieval>
- [11] **Pinterest (Graviton)**
<https://aws.amazon.com/solutions/case-studies/pinterest-graviton-case-study/>
- [12] **Honeycomb**
<https://www.honeycomb.io/blog/graviton-migration>
- [13] **Zomato**
<https://aws.amazon.com/blogs/opensource/category/big-data> (Zomato Trino/Druid on Graviton)
- [14] **Zendesk and other Graviton customers**
<https://aws.amazon.com/ec2/graviton/>
- [15] **Tinybird**
<https://www.tinybird.co/blog/how-we-cut-aws-costs-while-scaling-faster-with-eks-karpenter-and-spot-instances>
- [16] **Salesforce (Karpenter)**
<https://www.infoq.com/news/2026/01/salesforce-eks-karpenter>
- [17] **Moneysmart Group**
<https://aws.amazon.com/blogs/startups/cost-optimization-how-moneysmart-group-leveraged-aws-to-live-and-breathe-efficiency/>
- [18] **Database Savings Plans**
<https://aws.amazon.com/about-aws/whats-new/2025/12/database-savings-plans-savings>
- [19] **Prime Video**
<https://devclass.com/2023/05/05/reduce-costs-by-90-by-moving-from-microservices-to-monolith-amazon-internal-case-study-raises-eyebrows/>

- [20] Recall.ai**
<https://www.recall.ai/blog/how-websockets-cost-us-1m-on-our-aws-bill>
- [21] DoorDash**
<https://careersatdoordash.com/blog/staying-in-the-zone-how-doordash-used-a-service-mesh-to-manage-data-transfer-reducing-hops-and-cloud-spend/>
- [22] ECS Service Connect zone-aware routing**
<https://aws.amazon.com/blogs/containers/announcing-zone-aware-routing-in-amazon-ecs-service-connect/>
- [23] nOps (NAT gateway)**
<https://www.nops.io/reduce-nat-gateway-costs-using-nops-deep-insight-service>
- [24] 37signals**
<https://world.hey.com/dhh/our-cloud-exit-savings-will-now-top-ten-million-over-five-years-c7d9b5bd>
- [25] 37signals S3 exit**
<https://www.datacenterdynamics.com/en/news/37signals-begins-exiting-aws-storage-service/>
- [26] Dropbox S-1**
<https://www.sec.gov/Archives/edgar/data/0001467623/000119312518055809/d451946ds1.htm>
- [27] Dropbox savings analysis**
<https://www.datacenterknowledge.com/cloud/here-s-how-much-money-dropbox-saved-moving-out-cloud>
- [28] X/Twitter**
<https://www.datacenterdynamics.com/en/news/xtwitter-claims-100m-in-annual-savings-after-exiting-sacramento-data-center>
- [29] Prerender.io**
<https://levelup.gitconnected.com/how-we-reduced-our-annual-server-costs-by-80-from-1m-to-200k-by-moving-away-from-aws-2b98cbd21b46>
- [30] OneUptime**
<https://oneuptime.com/blog/post/2023-10-30-moving-from-aws-to-bare-metal/view>
- [31] GEICO**
<https://thenewstack.io/why-companies-are-ditching-the-cloud-the-rise-of-cloud-repatriation/>
- [32] GEICO interview**
<https://www.thestack.tech/the-big-interview-geico-vp-infrastructure-rebecca-weekly/>
- [33] S3 storage class prices**
<https://oneuptime.com/blog/post/2026-02-12-reduce-s3-storage-costs-lifecycle-policies/view>
- [34] Lambda on Graviton pricing**
<https://aws.amazon.com/blogs/aws/aws-lambda-functions-powered-by-aws-graviton2-processor-run-your-functions-on-arm-and-get-up-to-34-better-price-performance/>
- [35] NAT gateway vs endpoints**
<https://www.vantage.sh/blog/nat-gateway-vpc-endpoint-savings>